

Deep Learning Model based Human Activity Recognition using Kinect and Mobile Sensor

A Dissertation

Submitted in partial fulfillment of the requirements
for the award of Degree of

MASTER OF TECHNOLOGY

in

MECHANICAL ENGINEERING

by

(RISHIT MAURYA)

(Roll No.: 224ME1001)

Supervisor

Dr. RAVI KUMAR MANDAVA



**DEPARTMENT OF MECHANICAL ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DESIGN &
MANUFACTURING KURNOOL
KURNOOL (AP), INDIA**

April 2026

**DEPARTMENT OF MECHANICAL ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DESIGN &
MANUFACTURING KURNOOL
KURNOOL (AP), INDIA**



CERTIFICATE

This is to certify that the dissertation work entitled “**Deep Learning Model based Human Activity Recognition using Kinect and Mobile Sensor**”, which is being submitted by **Mr. Rishit Maurya** (Roll No. 224ME1001), is a bonafide work submitted to the Department of Mechanical Engineering, Indian Institute of Information Technology, Design & Manufacturing, Kurnool in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Mechanical Engineering.

To the best of our knowledge, the work incorporated in this thesis has not been submitted elsewhere for the award of any degree.

Dr. Ravi Kumar Mandava

Supervisor

Department of Mechanical Engineering

Indian Institute of Information Technology Design & Manufacturing

Kurnool-518008

Dr. Akhtar Khan

Head, Department of Mechanical Engineering

Indian Institute of Information Technology Design & Manufacturing

Kurnool-518008

APPROVAL SHEET

This Thesis entitled “**Deep Learning Model based Human Activity Recognition using Kinect and Mobile Sensor**” by **Rishit Maurya** is approved for the Degree of Master of Technology.

Examiners

Supervisor

Dr. Ravi Kumar Mandava

Head,

Department of Mechanical Engineering

DECLARATION

This is to certify that the work presented in the thesis entitled “**Deep Learning Model based Human Activity Recognition using Kinect and Mobile Sensor**” is a bonafide work done by me under the supervision of **Dr. Ravi Kumar Mandava** and was not submitted elsewhere for the award of any degree. I declare that this written submission represents my ideas in my own words and where others’ ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea / data / fact / source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Mr. Rishit Maurya

(Roll No.: 224ME1001)

ACKNOWLEDGMENT

Foremost, Praise to God.

Afterwards, I would like to express my sincere thanks and gratitude to my supervisors, **Dr. Ravi Kumar Mandava**, Asst. Professor. Mechanical Engineering Department, Indian Institute of Information Technology, Design & Manufacturing, Kurnool, for their continuous guidance, constructive suggestions, support, enthusiasm and motivation in my Master's Project work. It has been a benediction for me to spend many opportune moments under the guidance of the perfectionist at the aim of professionalism.

I am grateful to **Prof. BS Murty**, Director, Indian Institute of Information Technology, Design & Manufacturing, Kurnool and other top officials who gave me an opportunity to carry out research work. I thank **Dr. Akhtar Khan**, Head of the Department of Mechanical Engineering, for his help and continuous encouragement to complete this work.

I would like to acknowledge all the faculty members and staff of the Department of Mechanical Engineering for their valuable suggestions, cooperation, and assistance during the course of my research. My heartfelt thanks also go to my colleagues and friends for their encouragement, discussions, and moral support, which made this journey more meaningful and enjoyable.

Finally, I would like to express my deepest gratitude to my family for their unconditional love, patience, and continuous support, which have been the cornerstone of my achievements.

Mr. Rishit Maurya
(Roll No.: 224ME1001)

Abstract

Human Activity Recognition (HAR) plays a vital role in applications such as healthcare monitoring, smart environments, and human–robot interaction. This work presents a deep learning framework that jointly investigates wearable (smartphone inertial sensors) and non-wearable (Kinect-based skeletal data) modalities within a unified setup. A novel architecture, Multi-Scale Gated Residual Skip Bidirectional Long Short-Term Memory (MS-GRS-BiLSTM), is proposed to address limitations of conventional sequential models in capturing complex temporal dependencies.

The model combines multi-scale temporal feature extraction, gated residual skip connections, and bidirectional LSTM layers to effectively learn both short- and long-term motion patterns. Experiments were conducted on self-collected datasets and benchmark datasets including KARD, UCI-HAR, and mHealth.

For non-wearable skeletal data, the proposed model achieved accuracies of 98.35% on the self-collected dataset and 98.77% on KARD, significantly outperforming LSTM and BiLSTM baselines. For wearable sensor data, it consistently outperformed baseline models, achieving higher accuracy than LSTM (80.69%) and BiLSTM (86%+) while maintaining robustness to noise and motion variability. The model also demonstrates low inference time and compact size, enabling real-time deployment.

Overall, the proposed approach significantly improves HAR performance across both sensing modalities, providing an efficient and scalable solution for real-world applications.

Contents

Abstract	5
List of Figures	8
List of Tables	9
1 Introduction	1
2 Deep Learning Models	10
2.1 Long Short Term Memory (LSTM)	10
2.1.1 Motivation and Architecture	10
2.1.2 Mathematical Formulation	10
2.1.3 Application to Multimodal Sensor Data	12
2.2 Bidirectional LSTM (BiLSTM)	12
2.2.1 Motivation and Architecture	12
2.2.2 Application to Multimodal Sensor Data	13
2.3 Gated Residual Skip BiLSTM (GRS-BiLSTM)	14
2.3.1 Motivation: Addressing Multi-Scale Feature Dilution	14
2.3.2 Application to Multimodal Sensor Data	15
3 Proposed Methodology	16
3.1 Motivation for the Proposed Architecture	16
3.2 Architecture of the Proposed MS-GRS-BiLSTM Model	17
3.2.1 Multi-Scale Feature Extraction Module	17
3.2.2 Gated Residual Skip Connections	19
3.2.3 Bidirectional LSTM Layers	20
4 Non-Wearable Sensor Dataset	22
4.1 Dataset Overview	22
4.2 Data Collection Protocol	23
4.3 Dataset Characteristics and Preprocessing	25
4.3.1 Data Loading and Initial Processing	26
4.3.2 Temporal Segmentation via Sliding Windows	26

4.3.3	Feature Normalization	27
4.3.4	Label Encoding and Data Splitting	28
4.3.5	Multi-Scale Temporal Downsampling	28
4.4	Skeletal Dataset Results	29
4.5	KARD Dataset Results	32
5	Mobile Sensor Dataset	34
5.1	Dataset Overview	34
5.2	Data Collection Protocol	35
5.3	Dataset Characteristics and Preprocessing	36
5.3.1	Feature Engineering	37
5.3.2	Feature Normalization	38
5.3.3	Sliding Window Segmentation	38
5.3.4	Window Labelling via Majority Voting	39
5.3.5	Class Imbalance Handling	39
5.3.6	Train/Test Split	39
5.4	Mobile Dataset Results	40
5.5	M-Health Dataset Results	42
5.6	UCI Dataset Results	45
6	Prediction and Simulation	48
6.1	Real-Time Skeletal Tracking and Activity Prediction for non-wearable sensor	48
6.2	Simulation Results and Observations for non-wearable sensor	48
6.3	Real-Time Signal Monitoring and Activity Prediction for wearable sensor	50
6.4	Results and Observations of real-time prediction for wearable sensor	50
7	Conclusion	52
	References	53

List of Figures

2.1	LSTM Model Architecture	11
2.2	BiLSTM Model Architecture	13
2.3	GRS-BiLSTM Model Architecture	14
3.1	MS-GRS-BiLSTM Model Architecture	17
4.1	Volunteer performing Activity for Skeletal Dataset Collection	23
4.2	Skeletal Dataset Sample	24
4.3	Person vs Activity Distribution on Skeletal Dataset	25
4.4	Confusion Matrix on Skeletal Dataset	30
4.5	Accuracy/Loss curve over training on Skeletal Dataset	31
4.6	Confusion Matrix on KARD Dataset	32
4.7	Accuracy/Loss over epoch on KARD Dataset	33
5.1	Volunteer performing Activity for Mobile Dataset Collection	35
5.2	Mobile Dataset Sample	36
5.3	Person vs Activity Distribution on Mobile Dataset	37
5.4	Confusion Matrix on Mobile Dataset	41
5.5	Accuracy/Loss over epoch on mobile Dataset	42
5.6	Confusion Matrix on m-health Dataset	44
5.7	Accuracy/Loss over epoch on m-health Dataset	44
5.8	Confusion Matrix on UCI Dataset	46
5.9	Accuracy/Loss over epoch on UCI Dataset	46
6.1	Realtime prediction of activity using Kinect Sensor	49
6.2	Simulation & Prediction of activity using Kinect Sensor	49
6.3	Simulation & Prediction of activity using Mobile Sensor	51

List of Tables

4.1	Preprocessing Pipeline Summary	29
4.2	Classwise comparison of Accuracies over models on Skeletal Dataset	31
4.3	Metrics comparison over models on Skeletal Dataset	31
4.4	Metrics comparison over models on KARD Dataset	32
5.1	Wearable Sensor Preprocessing Pipeline Summary	40
5.2	Classwise comparison of Accuracies over models on Mobile Dataset	41
5.3	Metrics comparison over models on Mobile Dataset	42
5.4	Metrics comparison over models on mHealth Dataset	43
5.5	Metrics comparison over models on UCI Dataset	45

Chapter 1 - Introduction

Human Activity Recognition (HAR) is an interdisciplinary research field that aims to identify and classify physical movements through computational analysis of sensor-generated data. HAR systems decode patterns embedded in continuous sensor streams—such as triaxial acceleration signals or three-dimensional skeletal joint trajectories—and map them to semantically meaningful categories including walking, sitting, falling, or rehabilitation exercises. Although such recognition appears intuitive to humans, it is computationally challenging due to significant inter-subject variability arising from differences in age, body morphology, fitness level, and movement style.

The growing relevance of HAR is closely linked to demographic shifts, healthcare demands, and the development of intelligent environments. Applications range from elderly fall detection and rehabilitation monitoring to industrial safety, human–robot interaction, sports analytics, and smart home automation. The rapid expansion of IoT devices and embedded sensors—such as inertial measurement units (IMUs) and depth cameras—has generated unprecedented volumes of motion data, accelerating research in both algorithmic design and real-world deployment. A typical HAR pipeline involves signal preprocessing (filtering and sliding-window segmentation), feature extraction (handcrafted statistical descriptors or learned representations), and classification. Over time, classification methodologies have evolved from threshold-based rules and linear models to kernel methods such as Support Vector Machines (SVMs) and Random Forests, and more recently to deep learning approaches including Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks, CNN-LSTM hybrids, and attention-based models.

Human Activity Recognition has attracted sustained and growing research attention over the past two decades, producing a rich and diverse body of literature that spans classical machine learning, deep learning, wearable sensor systems, and vision-based approaches. The related work presented in this section is organized around two principal sensing paradigms employed in this research: wearable mobile sensor-based HAR and non-wearable depth-based HAR using the Microsoft Kinect sensor. Within each category, relevant studies are reviewed with attention to their methodological contributions, reported results, and identified limitations.

The transition from traditional machine learning to deep learning architectures represents one of the most consequential developments in wearable HAR research. Early systems relied

on handcrafted statistical features extracted from inertial sensor signals and paired with classifiers such as Support Vector Machines, k-Nearest Neighbors, and Random Forests. While these approaches demonstrated reasonable performance on constrained activity sets under controlled conditions, their dependence on manual feature engineering limited their scalability and generalizability. Two comprehensive surveys have documented this paradigm shift in considerable detail. Gu et al., in their work titled “A Survey on Deep Learning for Human Activity Recognition,” established that deep learning architectures eliminate the need for manual feature construction by learning hierarchical representations directly from raw sensor data. Their analysis identified generative models such as autoencoders as effective tools for dimensionality reduction and noise management, and acknowledged that multimodal and cross-subject generalization remain major open research challenges. Complementing this, Hossen and Abas, in “Machine Learning for Human Activity Recognition: State-of-the-Art Techniques and Emerging Trends,” traced the field’s evolution from classical methods toward emerging paradigms including federated learning approaches such as FedHAR, millimeter-wave radar, and Wi-Fi Channel State Information sensing, concluding that future systems must integrate edge computing capabilities and address ethical considerations surrounding privacy and algorithmic fairness. A further broad survey by the authors of “A Comprehensive Survey on Deep Learning Methods in Human Activity Recognition” reinforced these findings by identifying a clear shift from SVM and Random Forest approaches toward CNN and RNN architectures, while noting that persistent challenges including data scarcity, high annotation costs, and user reluctance to wear sensors continue to motivate interest in device-free recognition solutions.

Among deep learning architectures, CNNs and LSTMs have individually demonstrated strong performance on wearable HAR benchmarks, but their combination into hybrid architectures has proven particularly powerful. Khatun et al., in their study “Deep CNN-LSTM With Self-Attention Model for Human Activity Recognition,” proposed a hybrid model augmented with a self-attention algorithm and reported accuracies of 99.93% on a custom H-Activity dataset and 98.76% on the mHealth dataset. The self-attention component allowed the model to selectively emphasize the most diagnostically relevant temporal segments of the sensor signal while suppressing noise, making the system robust for clinical activity monitoring. Building on similar principles, the work titled “A Real-Time Deployable Attention-Driven CNN–LSTM Framework for Human Activity Recognition using Wearable Sensor” introduced a framework incorporating Convolutional Block Attention Modules and multi-resolution convolutional kernels, achieving 99.7% accuracy on the mHealth dataset with an inference latency of only 13.5 milliseconds. This result demonstrated that attention-guided learning can simultaneously satisfy laboratory-level accuracy requirements and the strict latency constraints of edge device deployment, effectively bridging the gap between controlled benchmark performance and real-world feasibility.

Hybrid architectures combining CNNs with bidirectional recurrent units have also drawn considerable research attention. A study introducing a “Novel CNN-BiLSTM-GRU Hybrid

Deep Learning Model for Human Activity Recognition” demonstrated that combining CNNs for local feature extraction with Bidirectional Gated Recurrent Units for capturing long-term temporal relationships produces a highly robust model targeted at bipedal robotics applications, achieving 99.7% accuracy on the WISDM dataset through careful optimization of batch size and network hyperparameters. The authors concluded that precise activity recognition is a prerequisite for meaningful human-robot interaction in both industrial and healthcare robotic contexts. Extending this architectural direction, the work on “A Multibranch CNN-BiLSTM Model for Human Activity Recognition Using Wearable Sensor Data” introduced parallel convolutional branches with different filter sizes that process the input signal simultaneously to capture temporal dependencies at multiple scales, achieving 96.05% on WISDM, 96.37% on UCI-HAR, and 94.29% on PAMAP2. The multibranch design proved particularly effective at managing the noise and systematic biases inherent in raw wearable sensor time-series data, since different filter sizes act as a bank of feature detectors sensitive to different temporal resolutions of motion.

Sain et al., in “Hybrid Deep Learning Model-Based Human Action Recognition in Indoor Environment,” investigated a CNN-BiLSTM model that incorporated hand-engineered kinematic features including joint velocity, acceleration, and height alongside learned representations, achieving 99.5% accuracy. Importantly, the model was demonstrated to be invariant to subject speed, clothing, and stance, properties that substantially enhance robustness in real-world rather than laboratory conditions. An alternative architectural direction was introduced by Ghafoor et al. in “Sensors-Based Human Activity Recognition Using Hybrid Features and Deep Capsule Network,” which argued that capsule networks retrieve equivariance properties that enhance spatial pattern recognition in sensor signals in ways that standard CNNs cannot replicate. Their DeepCapsNet model achieved accuracies of 99.9% on PAMAP2 and 98.5% on WISDM while simultaneously reducing computational cost relative to comparable deep architectures, making it an attractive option for resource-constrained deployment scenarios.

The challenge of limited labeled training data has motivated several lines of investigation into semi-supervised and generative approaches. Zilelioglu et al., in “Semisupervised Generative Adversarial Networks With Temporal Convolutions for HAR,” combined Generative Adversarial Networks with temporal convolutions to address the scarcity of annotated sensor data, reporting a 25% improvement over standard supervised methods when only approximately 3% of the training data was labeled. This result highlights the practical value of adversarial training for cross-subject scenarios in ambient assisted living environments, where large-scale annotation is impractical. In a complementary direction, Dahal et al. introduced the Stack-HAR framework in their paper “Stack-HAR: Complex Human Activity Recognition With Stacking-Based Ensemble Learning Framework,” employing a metalearning stacking ensemble that combined multiple base classifiers without requiring an extensive data preprocessing pipeline, demonstrating that ensemble strategies can effectively recognize complex activities in smart environment monitoring without the engineering overhead associated with deep learning

model design.

The degradation of recognition performance when transitioning from controlled laboratory settings to naturalistic deployment environments is a recurring concern in the wearable HAR literature. The study “Deep Learning Models for Real-Life Human Activity Recognition from Smartphone Sensor Data” found that performance dropped substantially in naturalistic conditions, with a hybrid depthwise separable CNN and LSTM architecture achieving a peak accuracy of 94.80% on real-life smartphone data—notably lower than figures typically reported on clean benchmark datasets. The authors concluded that automated feature selection through deep learning is more suitable for real-world smartphone data than manual feature engineering approaches, which tend to overfit the statistical regularities present in controlled recording sessions. Addressing the generalization challenge more directly, the work titled “Generalizable Sensor-Based Activity Recognition via Categorical Concept Invariant Learning” demonstrated that distribution shifts between different users are the primary driver of poor cross-subject HAR performance, and that the proposed CCIL framework substantially outperformed existing methods in cross-person and cross-dataset evaluation settings, establishing concept-invariant representation learning as an essential strategy for creating models that work accurately for new and previously unseen users.

Computational efficiency on resource-constrained mobile hardware has been approached from a novel training methodology perspective by Teng et al. in “The Layer-Wise Training Convolutional Neural Networks Using Local Loss for Sensor-Based HAR.” The authors argued that standard global loss backpropagation training induces backward locking that prevents efficient memory reuse on wearable devices, and demonstrated that local loss layer-wise training consistently outperformed global loss training across five benchmark HAR datasets while requiring significantly fewer parameters. This contribution is directly relevant to practical deployment on smartphones and embedded sensor platforms. Similarly, the work on “KNEE-HAR: A ConvLSTM-Based Lightweight Hybrid Deep-Learning Model for Human Activity Recognition With Knee Abnormalities” demonstrated that physiological sensor data including surface electromyography signals are highly effective for monitoring rehabilitation progress for lower-limb conditions, with ConvLSTM architectures efficiently extracting hidden patterns from raw physiological signals, and lightweight hybrid designs proving transformative for remote clinical monitoring applications.

The study by the authors of “Adaptive Gate Residual Connection and Multi-Scale RCNN for Fake News Detection” contributed relevant architectural insights, demonstrating that hybrid CNN-RNN models incorporating residual connections are highly effective at identifying sequential anomalies in balanced datasets and outperforming standard Bi-LSTM-GRU models on sustainability and accuracy metrics, though the authors noted that deep sequential models frequently struggle with explainability—a limitation shared with many HAR deep learning systems. Transfer learning as a strategy for improving generalization across datasets was examined by Ray et al. in “Transfer Learning Enhanced Vision-Based Human Activity Recognition: A

Decade-Long Analysis,” which analyzed transfer learning applications on benchmark datasets including UCF-101 and HMDB-51 and found that performance is directly proportional to the relatedness between source and target datasets, concluding that pairing deep learning with transfer learning is essential for constructing robust, generalizable human action recognition systems suitable for a smart society.

The investigation by Sarveshwaran et al., published as “Investigation on Human Activity Recognition using Deep Learning,” confirmed that deep learning models consistently outperform traditional Artificial Neural Networks for complex activities such as dancing, with view-based models using binary motion images performing well on both 2D and 3D datasets. Despite these advances, the authors emphasized that real-world sensor noise remains a significant challenge for existing HAR models that perform well in controlled environments.

The Microsoft Kinect sensor has occupied a central position in non-wearable HAR research since its introduction, owing to its capacity to generate real-time three-dimensional skeletal joint data without requiring any instrumentation of the observed subject. Its combination of depth sensing, inherent privacy preservation through skeletal abstraction, and relatively low cost has made it an attractive platform for a wide range of activity recognition, gait analysis, posture monitoring, and rehabilitation assessment applications.

Several foundational studies have examined the practical capabilities and limitations of the Kinect sensor before addressing activity recognition specifically. The work titled “An Analysis of Kinect-Based Human Fall Detection System” determined that the sensor performs most reliably at a working distance of 3 to 3.5 meters under a light intensity of approximately 1007 lux, successfully identifying the correct human posture status for all tested cases under these optimal conditions, and concluding that Kinect-based monitoring provides a low-cost and reliable solution for elderly fall detection without the need for wearable sensors. A complementary evaluation presented in “Evaluation of Kinect 3D Sensor for Healthcare Imaging” found that Kinect joint tracking accuracy under controlled conditions is comparable to expensive marker-based motion capture systems, though errors were most frequent during sitting postures where self-occlusion of body parts such as crossed legs causes tracking failures—a finding with important implications for activity recognition systems that include seated activities in their target class set. Milovanovic et al., in “Kinect v2 Tracked Body Joint Smoothing for Kinematic Analysis in Musculoskeletal Disorders,” identified erratic jitter in three-dimensional joint coordinates arising from occlusion and field-of-view limits as a significant obstacle to reliable kinematic analysis, and proposed a smoothing filter that achieved correlations of 0.95 to 0.98 with gold-standard goniometry measurements, demonstrating that the resulting motion signatures are effective for evaluating musculoskeletal treatment outcomes.

Activity recognition using Kinect skeletal data has been pursued through a wide variety of feature engineering and classification strategies. Le et al., in “Human Posture Recognition Using Human Skeleton Provided by Kinect,” demonstrated that relative joint angles are substantially more robust features than absolute joint coordinates when the subject’s position or

orientation relative to the sensor changes, achieving an average accuracy of 98.2% for four fundamental postures—lying, sitting, standing, and bending—in a health monitoring framework. The authors emphasized that data scaling prior to SVM classification is critical for maintaining high recognition rates under varying subject positions. Komang et al., in “Human Activity Recognition Using Skeleton Data and Support Vector Machine,” investigated the effect of subject-to-sensor distance on Kinect-based HAR and found that 2 meters represents the optimal operating distance for reliable joint tracking, with performance averaging 93.75% at this distance but dropping significantly at 1 meter due to partial loss of tracked joint information. Ashwini et al., in their “Skeletal Data Based Activity Recognition System,” proposed using Principal Component Analysis to eliminate redundant features from statistical skeletal joint descriptors before SVM classification, achieving 97% accuracy and outperforming KNN, with the authors concluding that feature dimensionality reduction is crucial for building efficient healthcare and assisted living surveillance systems.

The use of simplified skeletal representations has attracted attention as a strategy for reducing computational complexity while preserving recognition accuracy. Reddy and Chattopadhyay, in “Human Activity Recognition from Kinect Captured Data Using Stick Model,” identified the head and shoulder joints as the most reliable and noise-free landmarks in the Kinect skeletal model and demonstrated that using only four major joint points for feature extraction achieved 97.29% accuracy while substantially reducing time complexity relative to full skeleton approaches. The authors proposed this simplified stick model as a practical and privacy-preserving approach for health monitoring of elderly individuals in bedroom settings. Kumar and Kala, in “Human Activity Recognition in Real-Time Environments Using Skeleton Joints,” demonstrated that using only 10 to 15 percent of the total frames in a video sequence is sufficient for training an efficient real-time recognition system using Eigen joint action descriptors and distance-based classifiers, achieving accuracies of 94% to 98% across tested activity classes—a finding directly relevant to real-time applications such as gesture recognition in interactive video games and human-computer interfaces.

Deep learning applied to Kinect skeletal data has produced some of the most competitive results in the non-wearable HAR literature. Yadav et al., in “Skeleton-Based Human Activity Recognition Using ConvLSTM and Guided Feature Learning,” proposed a sequential fusion of CNNs and LSTMs for spatiotemporal feature extraction from skeletal data and achieved 98.89% accuracy on the KinectHAR dataset comprising 130,000 samples, emphasizing that the use of derived skeletal features rather than raw video fully preserves subject privacy. Açış and Güney, in “Classification of Human Movements by Using Kinect Sensor,” found that skeletal joint data substantially outperforms raw RGB image data for identifying posture disorders, with an LSTM-SVM hybrid approach achieving 98.95% accuracy. Their analysis noted that learning-from-scratch methods applied to RGB data require exceptionally large datasets, whereas extracting specialized joint features is far more efficient and practical for posture analysis tasks. For gait-specific recognition, Sepas-Moghaddam et al. introduced the KinectGaitNet architecture

in their paper “KinectGaitNet: Kinect-Based Gait Recognition Using Deep Convolutional Neural Network,” employing global average pooling to accommodate variable-length gait cycles without data resampling and achieving 99.47% accuracy on gait datasets using nine times fewer parameters than comparable state-of-the-art models, providing a fast and robust approach to person identification through gait.

Graph-based skeletal representations have emerged as a particularly principled and theoretically motivated approach to Kinect-based activity recognition. The paper “An Investigation of GCN-Based Human Action Recognition Using Skeletal Features” established that human skeletal data is inherently graph-structured, with joints as nodes and anatomical connections as edges, making Graph Convolutional Networks more expressive than standard CNNs for this type of data. The study demonstrated that GCN-based recognition is insensitive to lighting variations and recommended that future research explore directed acyclic graph formulations to better capture the directional flow of motion information between body segments. A related hybrid approach was presented in “GCN-LSTM: A Hybrid Graph Convolutional Network Model for Schizophrenia Classification,” where GCN-LSTM models were applied to graph-structured EEG signal data, achieving a classification rate of 99.25% by extracting time-frequency features from graph nodes. While primarily targeting psychiatric classification rather than HAR directly, this work demonstrated the generalizability and power of combining graph-based spatial modeling with recurrent temporal modeling—an architectural principle equally applicable to skeletal joint sequence analysis.

Continuous and real-time activity recognition presents a distinct set of challenges beyond those associated with the classification of isolated, pre-segmented action sequences. The paper “A Novel Framework of Continuous Human-Activity Recognition Using Kinect” addressed this by proposing a coarse-to-fine recognition strategy that first groups activities into broad categories such as sitting versus standing before performing fine-grained classification, achieving 68.9% accuracy across 24 distinct activities and outperforming traditional isolated recognition approaches. The authors argued that continuous HAR is inherently more representative of real-world scenarios than models designed to classify only cleanly segmented single-action sequences. Zhu et al., in “An Online Continuous Human Action Recognition Algorithm Based on the Kinect Sensor,” modeled continuous actions as ordered sequences of key poses and atomic motions and developed an online recognition system that requires no prior detection of action start and end points, using variable-length Maximum Entropy Markov Models to achieve both recognition efficiency and effectiveness in human-robot interaction contexts.

Bari and Gavrilova, in “Artificial Neural Network Based Gait Recognition Using Kinect Sensor,” introduced two novel geometric features—joint relative cosine dissimilarity and joint relative triangle area—that are invariant to viewpoint and pose changes, and demonstrated that a deep learning neural network using these features outperformed traditional KNN and SVM classifiers for unobtrusive person authentication through gait analysis, enabling identity verification without requiring direct user interaction. Ren et al., in “Human Posture Recognition

Using a Hybrid of Fuzzy Logic and Machine Learning Approaches,” addressed the fine-grained problem of lying posture recognition and argued that fuzzy logic effectively manages the uncertain and continuously varying dynamics of human limb movement during bed-based activities, achieving 97.1% accuracy across 20 distinct lying postures relevant to autonomous assistive robotic caregiving. The challenge of self-occlusion in Kinect-based motion capture was directly addressed in the research titled “Human Upper-Body Motion Capturing Using Kinect,” where a hybrid classifier combining Adaboost and Random Forest with offset compensation was demonstrated to outperform the standard Microsoft Kinect SDK in recovering joint positions hidden from the camera, confirming that self-occlusion is the primary technical obstacle in capturing articulated human motion with a single depth sensor.

Specialized applications of Kinect-based HAR have extended into interactive training and exercise monitoring contexts. Kumar et al., in “Virtual Trainer with Real-Time Feedback Using Kinect Sensor,” developed a system that provides real-time matching scores and corrective visual feedback during exercise sessions, achieving 96% accuracy in user-independent training mode for nine basic exercise postures. The authors described this as a meaningful contribution to human-computer interaction that enables correct exercise technique without requiring a human trainer. Finally, the study by Sarveshwaran et al. titled “Investigation on Human Activity Recognition using Deep Learning” examined view-based deep learning models using binary motion images and confirmed their effectiveness across both 2D and 3D activity datasets, while noting that real-world sensor interference remains a persistent challenge even for architectures that perform strongly under controlled Kinect-based recording conditions.

The body of literature reviewed in this section reveals that both wearable inertial sensor-based and non-wearable Kinect-based approaches to HAR have advanced considerably, with deep learning architectures—particularly CNN-LSTM hybrids, attention-driven models, graph convolutional networks, and capsule networks—achieving strong benchmark performance across diverse activity sets and evaluation datasets. Surveys by Gu et al., Hossen and Abas, and the authors of the comprehensive deep learning HAR survey collectively confirm that the field is moving toward more generalizable, computationally efficient, and privacy-respecting systems, while identifying cross-subject generalization, data scarcity, and real-world deployment as persistent open problems.

On the wearable side, studies including those by Khatun et al., Zilelioglu et al., and the CCIL framework authors highlight that controlled benchmark accuracy does not always translate to real-world performance, and that inter-subject variability remains a fundamental limitation. On the non-wearable Kinect side, contributions from Reddy and Chattopadhyay, Yadav et al., and Açış and Güney demonstrate that skeletal representations support strong recognition performance while preserving privacy, though sensor-specific constraints including operating range, self-occlusion sensitivity, and indoor-only suitability limit generalizability. Despite the breadth of these contributions, few existing studies have systematically evaluated and compared both sensing modalities within a unified experimental framework. This research addresses

that gap by developing and evaluating HAR systems grounded in both wearable smartphone-based inertial sensing and non-wearable Kinect-based skeleton tracking, providing comparative empirical insights into their respective strengths, limitations, and complementary potential across a shared set of activity classes and evaluation conditions.

Chapter 2 - Deep Learning Models

Human Activity Recognition (HAR) depends on strong temporal modeling to truly understand how movements flow over time. In this study, I compare three deep learning models using two kinds of sensor data: wearable sensors that is smartphone accelerometer and gyroscope data captured with the Sensor Logger app and non-wearable, depth-based skeletal tracking collected from Microsoft Kinect. The models I look at are Long Short-Term Memory (LSTM), Bidirectional LSTM (BiLSTM), and the new Gated Residual Skip BiLSTM (GRS-BiLSTM) I've developed for this work. Each of these models tackles different, increasingly tough problems in modeling the sequence of human activities.

2.1 Long Short Term Memory (LSTM)

2.1.1 Motivation and Architecture

Standard Recurrent Neural Networks (RNNs) really struggle to hold onto information for any meaningful length of time. Their gradients either vanish or explode during backpropagation, so they just can't grasp patterns that play out over several seconds. That's a real headache in Human Activity Recognition (HAR), where distinguishing between walking and jogging isn't just about immediate movements—it's about picking up on those subtle, longer-term cues. The LSTM architecture, developed by Hochreiter and Schmidhuber back in 1997, changed the game. Instead of relying on unreliable memory, LSTMs use memory cells and gates to decide which bits of information to keep, forget, or update as the sequence unfolds. This way, the model hangs onto the right context and escapes those gradient problems.

The impact here is huge. LSTMs don't just blindly pass information forward and suffer from gradient decay. These networks get picky—they remember what's important, forget what's not, and adapt to the nuances in temporal data. This selective memory lets them learn which features are actually useful for identifying activities, so their performance in HAR just leaps ahead.

2.1.2 Mathematical Formulation

Given an input sequence $\mathbf{x} = \{x_1, x_2, \dots, x_T\}$ where $x_t \in \mathbb{R}^d$, the LSTM computes:

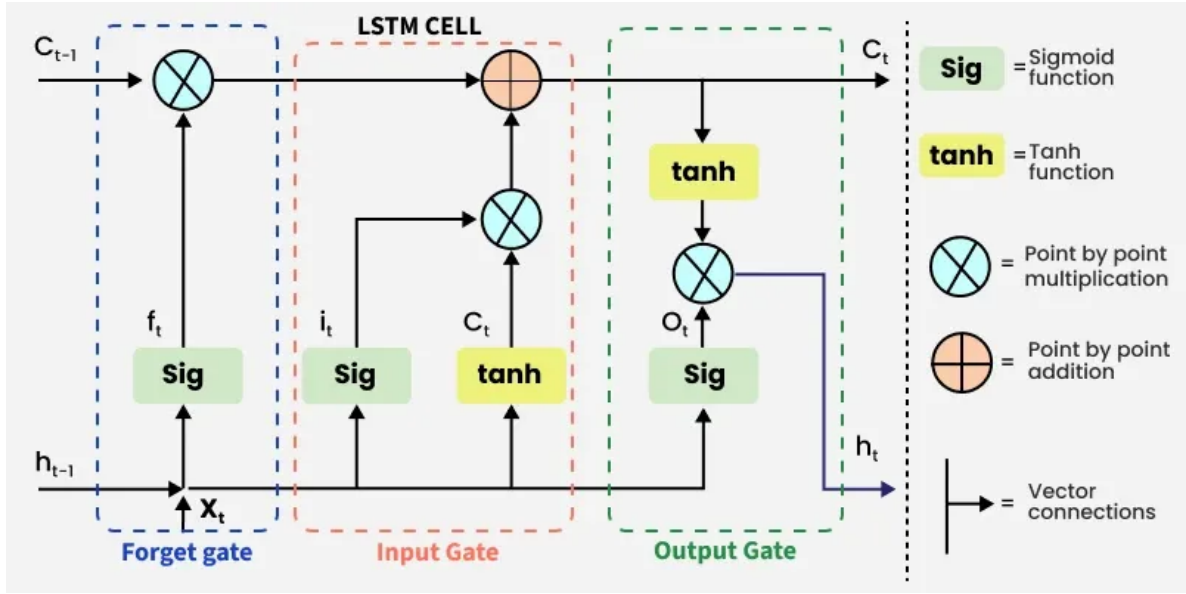


Figure 2.1: LSTM Model Architecture

Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.1)$$

Input Gate

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.2)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (2.3)$$

Cell State Update

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (2.4)$$

Output Gate

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.5)$$

$$h_t = o_t \odot \tanh(C_t) \quad (2.6)$$

Symbol Definitions Here, x_t denotes the input at time t , h_t represents the hidden state at time t , and h_{t-1} is the previous hidden state. The cell state at time t is denoted by C_t , with C_{t-1} representing the previous cell state. The variables f_t , i_t , and o_t correspond to the forget gate, input gate, and output gate activations, respectively, while $\tilde{C}_t$ denotes the candidate cell state. The matrices W_f , W_i , W_C , and W_o represent the weight parameters associated with the forget, input, cell, and output gates, and b_f , b_i , b_C , and b_o denote their respective bias vectors. The function $\sigma(\cdot)$ denotes the sigmoid activation function, $\tanh(\cdot)$ represents the hyperbolic tangent

function, $\odot$ indicates element-wise multiplication, and $[\cdot, \cdot]$ denotes the concatenation operator. Finally, T represents the sequence length and d denotes the input dimensionality.

The additive cell state update is crucial, as gradients propagate through the cell state with only multiplicative gate interactions, avoiding repeated matrix multiplications and effectively addressing the vanishing gradient problem.

2.1.3 Application to Multimodal Sensor Data

Wearable Data: Input consists of windowed segments where $x_t \in \mathbb{R}^6$ encodes triaxial accelerometer and gyroscope readings. LSTM works by moving through sequences from left to right, gathering context as it goes. This helps it track how things unfold over time—like building up before a jump or recognizing the repeating rhythm in a walk. The forget gate plays a key role here, wiping out old activity signals once a movement phase ends, so the model can shift cleanly to whatever comes next. But since it only looks forward, not backward, any subtle hints that show up later in the sequence can’t influence how it interprets earlier steps.

Kinect Skeletal Data: Input $x_t \in \mathbb{R}^{3J}$ (J tracked joints) encodes complete body pose configurations. Activities show up as coordinated limb movements, like waving, or as changes in posture, such as moving from sitting to standing. This is a built-in limitation that later models tackle differently.

2.2 Bidirectional LSTM (BiLSTM)

2.2.1 Motivation and Architecture

Why BiLSTM? Standard LSTM’s unidirectional processing constrains the hidden state h_t to encode only past observations $\{x_1, \dots, x_t\}$, ignoring future context $\{x_{t+1}, \dots, x_T\}$. In HAR, complete temporal context is often discriminative—distinguishing a preparatory arm swing during walking from the initial phase of a fall event requires information from subsequent time steps.

How it changes modeling: BiLSTM processes sequences simultaneously in both temporal directions through independent forward (LSTM) and backward (LSTM) networks, then combines representations:

Forward and Backward Passes

$$\vec{h}_t = \vec{\text{LSTM}}(x_t, \vec{h}_{t-1}, \vec{C}_{t-1}), \quad h_t = \text{LSTM}(x_t, h_{t+1}, C_{t+1}) \quad (2.7)$$

Combined Representation

$$h_t^{\text{bi}} = [\vec{h}_t \parallel h_t], \quad h^{\text{final}} = [h_T \parallel h_1] \quad (2.8)$$

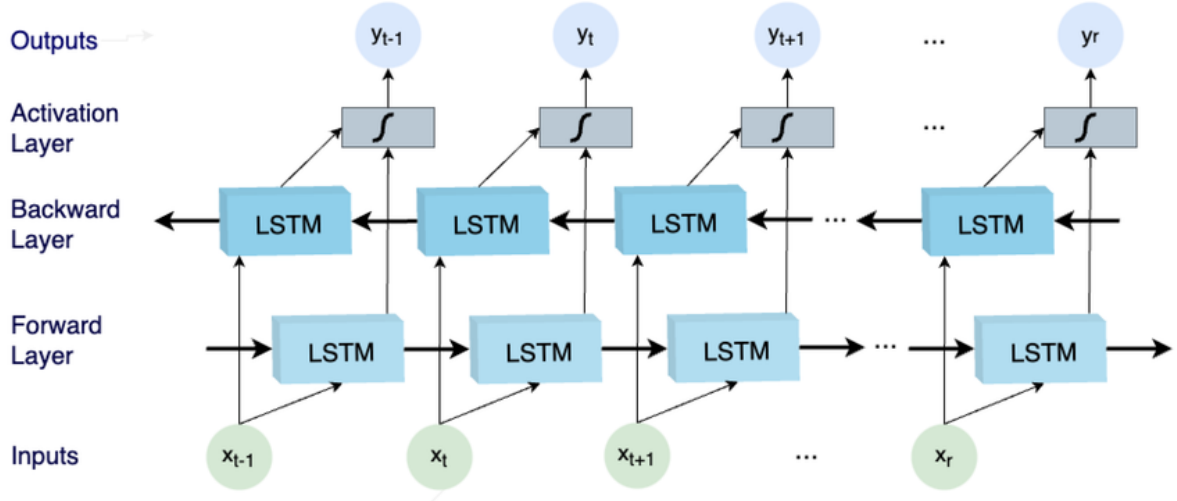


Figure 2.2: BiLSTM Model Architecture

Classification

$$\hat{y} = \text{softmax}(W_c \cdot h^{\text{final}} + b_c) \quad (2.9)$$

Symbol Definitions $\vec{h}_t$ = forward hidden state at time t , h_t = backward hidden state at time t , $\vec{C}_t$ = forward cell state, C_t = backward cell state, h_t^{bi} = bidirectional hidden state at time t , h^{final} = final sequence representation, $\parallel$ = concatenation operator, $\hat{y}$ = predicted class probabilities, W_c = classification weight matrix, b_c = classification bias vector, T = sequence length, H = number of hidden units per direction.

Impact: Each time step's representation now encodes bidirectional context both antecedent and consequent temporal information fundamentally improving discriminative capacity for ambiguous activity phases.

2.2.2 Application to Multimodal Sensor Data

Wearable IMU Data: The BiLSTM makes a real difference with activities that look alike at first, like stumbling versus falling. With the backward LSTM, the model captures how things end and uses that information to better understand what happened earlier.

Kinect Skeletal Data: The benefit gets even clearer here. Movements like squats or gestures don't just flow in one direction. The most telling skeletal positions come in the middle, but they only really make sense if you know what came before and what comes after. The backward LSTM learns how the body moves toward these postures—the “inverse dynamics”—and, when you combine this with the usual forward perspective, you get a full, detailed snapshot of each activity in context. This comprehensive view makes it possible to tell apart actions that look almost identical at their midpoint, like sitting down versus standing up.

2.3 Gated Residual Skip BiLSTM (GRS-BiLSTM)

2.3.1 Motivation: Addressing Multi-Scale Feature Dilution

Why propose GRS-BiLSTM? Standard BiLSTMs capture context in both directions, but when you stack them deeper—necessary for learning hierarchical temporal patterns—you run into a problem. As the network grows, subtle, lower-level temporal features start to fade away. They get weakened as the data passes through layer after layer of nonlinear transformations. Basic stacked BiLSTMs don't have a good way to control which features from which layers actually matter at the end.

Core Idea: We bring in two key mechanisms. First, residual skip connections let information from early layers flow directly to deeper ones without getting lost. Second, a learned gating system decides, on the fly, how much of this skip information actually influences the output. By combining ideas from residual learning (He et al.) and highway networks, we adapt these strategies to work with bidirectional recurrent networks for multi-modal human activity recognition.

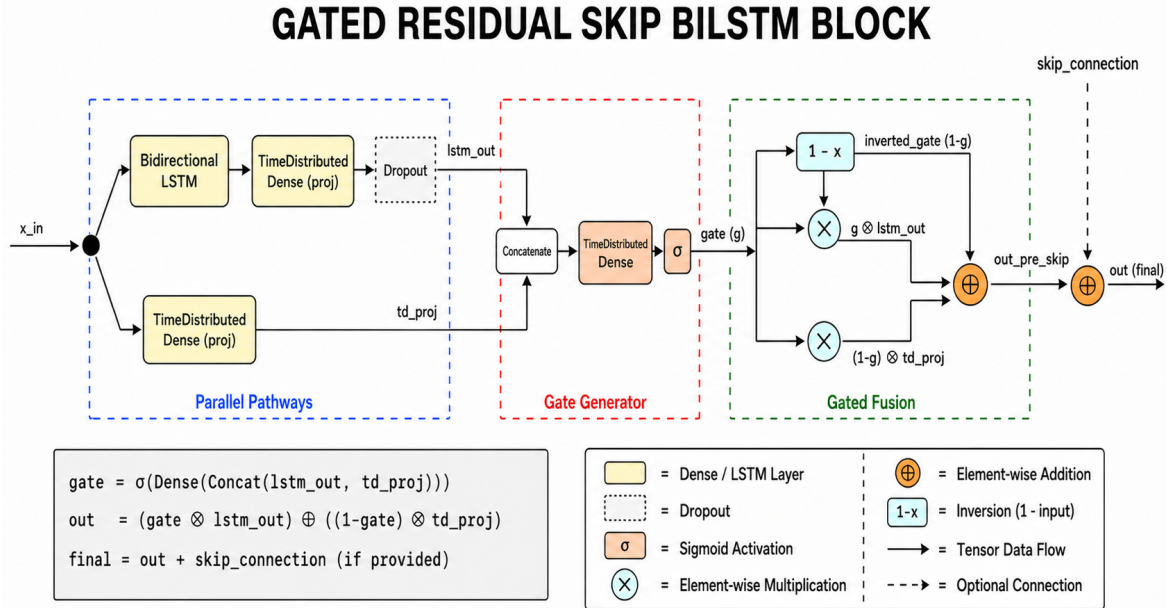


Figure 2.3: GRS-BiLSTM Model Architecture

Dimensionality Projection

$$x_{\text{proj},t}^1 = x_t^1 \cdot W_{\text{proj}}^T + b_{\text{proj}} \quad (2.10)$$

Gate Computation

$$g_t = \sigma \left(x_t^2 \cdot W_g^T + b_g \right) \quad (2.11)$$

Complete GRS Block (Unified Expression)

$$\tilde{h}_t = x_t^2 + g_t \odot x_{\text{proj},t}^1 \quad (2.12)$$

Symbol Definitions $\tilde{h}_t \in \mathbb{R}^{256}$ = gated residual output at time t , $x_t^1 \in \mathbb{R}^{512}$ = first BiLSTM layer output (low-level features), $x_t^2 \in \mathbb{R}^{256}$ = second BiLSTM layer output (high-level features), $x_{\text{proj},t}^1 \in \mathbb{R}^{256}$ = projected first layer output, $W_{\text{proj}} \in \mathbb{R}^{256 \times 512}$ = projection weight matrix for dimensionality alignment, $b_{\text{proj}} \in \mathbb{R}^{256}$ = projection bias vector, $g_t \in (0, 1)^{256}$ = adaptive gate vector, $W_g \in \mathbb{R}^{256 \times 256}$ = gate weight matrix, $b_g \in \mathbb{R}^{256}$ = gate bias vector, $\sigma(\cdot)$ = sigmoid activation function, $\odot$ = element-wise multiplication, $g_t^{(j)}$ = j -th element of gate vector controlling the j -th feature dimension.

How the gating mechanism works: When $g_t^{(j)} \approx 1$, the j -th feature from Layer 1 flows right through, so you hang on to all those subtle details. When $g_t^{(j)} \approx 0$, that skip connection fades and Layer 2 takes over for that dimension. Critically, x_t^2 always stays in the mix. The skip connection just adds a learned tweak, it never replaces what Layer 2 produces.

This design changes the way the network learns. First, it creates direct paths for the gradients, so you don't get stuck with vanishing signals as you move deeper. Second, it lets the model blend information at different scales, adjusting on the fly for every feature and every time step. Last, since it ties deeper layers back to the details at lower levels, it adds a sort of natural regularization. That keeps things grounded and helps prevent overfitting.

2.3.2 Application to Multimodal Sensor Data

Wearable IMU Data: Inertial signals aren't just noise — they actually capture things happening at different speeds. When models don't connect these layers, you lose the middle ground; that essential detail just slips away with each data transformation.

GRS-BiLSTM fixes this. It builds direct routes for those lower-level signals to reach the final output. The smart gates inside the model know when to highlight brief impact spikes and when to draw out longer, steadier patterns. This means the system gets better at telling activities apart, especially those that look the same on a big scale but are really different when you zoom in — like walking versus jogging, or going up stairs versus down.

Kinect Skeletal Data: These skeletal paths track everything from split-second joint angles to bigger movement chunks and the full activity from start to finish. With gating, pose-level details stick around and don't get washed out when the model zooms out to spot the whole activity.

And because Kinect data is pretty clean and organized, the gated skip helps stop the model from latching onto random, fake patterns at higher levels. By always tying deeper layers back to the basics — those clear skeletal features — GRS-BiLSTM keeps its multi-level perspective. That way, it stays reliable even when people move differently from one another.

Chapter 3 - Proposed Methodology

Most sequential models struggle with the messiness and unpredictability of human activity data. To tackle these problems head-on, we introduce a new architecture: the Multi-Scale Gated Residual Skip Bidirectional LSTM, or MS-GRS-BiLSTM. This model isn't just another tweak on LSTMs—it stitches together three key ideas. First, it uses multi-scale feature extraction with structured downsampling, so it pulls information from data at different time scales. Second, it builds in gated residual connections to keep the flow of information under control as signals move between layers, preventing the usual issues with vanishing or exploding gradients. Third, it employs Bidirectional LSTM layers, letting the network understand sequence data in both forward and backward directions for a fuller picture. All of these pieces work together in one seamless, trainable system. The result? The model catches the important signals over both short and long timeframes, keeps gradients stable, and really digs into the complicated patterns behind human activities.

3.1 Motivation for the Proposed Architecture

Human activities always play out across different time scales. Take walking: you've got quick, small movements like each foot hitting the ground, and at the same time, there are larger, slower patterns—your overall stride and rhythm. Most LSTM models are pretty good at handling sequences, but they usually focus on just one scale at a time. That means they miss out on important details happening at other time scales.

If you try to make these models deeper to catch more complex patterns, you run into other problems. The further features move through the layers, the more watered down or lost they get—a problem known as gradient degradation and feature dilution. Residual connections are supposed to help with this, letting information skip ahead and avoid getting lost. But standard residuals aren't picky; they just pass everything through, whether it's useful or not.

That's where gated residual connections come in. Instead of blindly skipping information ahead, they use a learnable gate to decide which data to keep and which to drop, layer by layer.

One more thing: regular unidirectional LSTMs only process data forward in time. That means they're blind to future context, which is often the key to telling similar activities apart. The new architecture fixes all of these issues with three main components, each tackling a

specific limitation. You'll find details on each of them in the next sections.

3.2 Architecture of the Proposed MS-GRS-BiLSTM Model

This architecture brings together three main pieces: First, a Multi-Scale Feature Extraction Module that uses structured downsampling to pull features at different scales. Second, Gated Residual Skip Connections that decide what information to keep as data moves through the layers. And third, Stacked Bidirectional LSTM Layers that dig into the sequence from both directions to really capture how things relate over time.

MULTI-SCALE GATED RESIDUAL SKIP BiLSTM ARCHITECTURE

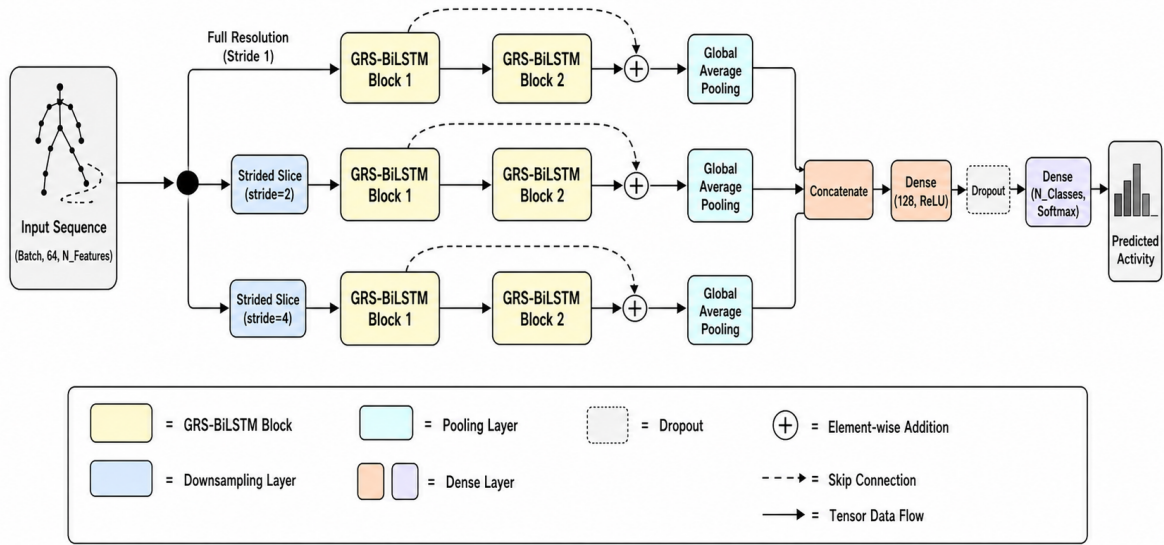


Figure 3.1: MS-GRS-BiLSTM Model Architecture

3.2.1 Multi-Scale Feature Extraction Module

The multi-scale feature extraction module sits at the heart of this architecture. Instead of sticking to just one temporal resolution, it splits the input sequence into three branches, each handling a different timescale. These scales come from specific downsampling factors: 0, 2, and 4.

Human activities don't all play out at the same speed. Important details pop up at different timescales. No single resolution gives you the full picture. You need fine details to catch quick changes—like sudden acceleration or fast limb movements—and you also need broader, slower patterns to understand the bigger shape of what someone's doing. By running the input through three parallel branches at once, the model grabs onto features across the full time range.

Let's break down the branches:

1. Branch 1 doesn't downsample at all—it takes the input as-is. This keeps all the high-frequency details. So anything fast or short, like quick impacts or subtle limb flicks, stays

right there in the data.

2. Branch 2 downsamples by a factor of 2, so it cuts the temporal resolution in half. This one homes in on patterns that last a bit longer—like the repeating rhythm of an activity or transitions between movements.
3. Branch 3 goes further, downsampling by 4. Here, you get a broad look at the whole activity, perfect for picking up on long-running movement structures or the general shape of what’s happening. It also helps filter out fast, noisy details you don’t want when you’re just after the big picture.

Altogether, these three branches guarantee the model sees both the fine and the coarse details, giving a thorough read on any kind of motion in the input. Formally, for each branch $s \in \{1, 2, 3\}$ with corresponding downsampling factor $d_s \in \{1, 2, 4\}$, the downsampled input is obtained as:

$$X^{(s)} = \text{Downsample}(X, d_s) \quad (3.1)$$

where $X^{(s)} \in \mathbb{R}^{\lfloor T/d_s \rfloor \times C_{in}}$ denotes the temporally downsampled input at scale s , T is the original sequence length, and C_{in} is the number of input channels. Each branch then applies a 1D Convolutional layer to extract local temporal features from the downsampled sequence:

$$F_s = \text{ReLU}(\text{Conv1D}(X^{(s)}; W_s, b_s)) \quad (3.2)$$

where $F_s \in \mathbb{R}^{T_s \times C}$ is the resulting feature map, $T_s = \lfloor T/d_s \rfloor$ is the temporal length at scale s , C is the number of convolutional filters, and W_s, b_s are the learnable parameters specific to branch s . The ReLU activation introduces non-linearity, enabling each branch to detect complex local motion patterns characteristic of its respective temporal scale. Each branch independently learns its own set of filters, ensuring that the features extracted at the three scales are complementary rather than redundant.

To enable fusion across branches with differing temporal lengths, the feature maps from all three branches are projected to a uniform temporal dimension T_{out} through adaptive temporal pooling:

$$\tilde{F}_s = \text{AdaptivePool}(F_s, T_{out}) \quad (3.3)$$

The temporally aligned feature maps are then concatenated along the feature dimension to produce a unified multi-scale representation:

$$F_{\text{multi}} = [\tilde{F}_1 \parallel \tilde{F}_2 \parallel \tilde{F}_3] \quad (3.4)$$

where $\parallel$ denotes channel-wise concatenation and $F_{\text{multi}} \in \mathbb{R}^{T_{out} \times 3C}$. This fused representation encodes activity-relevant information simultaneously from fine, intermediate, and coarse

temporal scales into a single unified feature space that is subsequently processed by the BiLSTM layers.

3.2.2 Gated Residual Skip Connections

When multi-scale features move through several BiLSTM layers, there's a real chance that the finer motion details—especially those from short time scales—get lost along the way. Each layer's non-linear transformations can drown out or distort those subtle signals. To fight this, the model uses Gated Residual Skip Connections between its layers.

Standard skip connections just add the input of a layer to its output, no questions asked. Gated residual connections do more. They use a learnable gate vector to decide, on the fly, how much of that input should actually make it through the skip connection. This lets the network hold on to key details from earlier layers, while still letting deeper layers build up new and more complex representations.

Formally, let $h^{(l)}$ denote the output of the l -th BiLSTM layer and $h^{(l-1)}$ denote its input (or the output of the preceding layer). The gated residual connection is computed as:

$$g^{(l)} = \sigma \left(W_g \cdot [h^{(l)}, h^{(l-1)}] + b_g \right) \quad (3.5)$$

$$\tilde{h}^{(l)} = g^{(l)} \odot h^{(l)} + (1 - g^{(l)}) \odot h^{(l-1)} \quad (3.6)$$

where $g^{(l)} \in \mathbb{R}^{2H}$ is the gate vector computed from the concatenation of the current and previous layer outputs; σ is the sigmoid activation function that constrains the gate values to the range 0–1; W_g and b_g are learnable parameters of the gating mechanism; and $\odot$ denotes element-wise multiplication. The resulting $\tilde{h}^{(l)}$ is a weighted interpolation between the current layer output and the residual from the previous layer, where the interpolation weights are adaptively determined by the gate $g^{(l)}$.

When gate values get close to 1, the model leans heavily on the new features learned in the current layer. But when those values drop toward 0, the model skips the current transformation and just passes along what it got from the previous layer. This kind of flexibility lets the network decide for itself how much it should change the features at each step. It also helps protect the important details picked up in earlier layers from getting messed up by unnecessary transformation.

These gated residual connections actually do a couple more things in this architecture. First, they keep gradients flowing smoothly through the layers. Since gradients can disappear by the time they reach the start of a deep recurrent network, having these gates gives gradients a shortcut, which makes training more reliable and less frustrating. Second, the gating system helps the network mix information from different layers. This means the final sequential representation gets to use both the fine-grained motion details and the bigger-picture activity

patterns, instead of losing either one along the way.

3.2.3 Bidirectional LSTM Layers

The multi-scale fused representation F_{multi} , goes through gated residual connections for some initial clean-up. Unlike regular LSTMs that only look at the sequence from start to finish, BiLSTMs check both directions at once—so at every time step, the model gathers information from the past and the future. This really matters when it comes to recognizing human activities. Getting a motion right usually depends on knowing what just happened and what comes next. Take, for example, someone moving from sitting to standing or the other way around. If you only look at the movement in the middle, both transitions might seem pretty much the same. But when the model can see what movements led up to that point and what happens after, it becomes much better at figuring out which action it is actually seeing.

At each time step t , the BiLSTM computes:

Forward direction

$$\vec{h}_t = \text{LSTM}(F_{\text{multi},t}, \vec{h}_{t-1}) \quad (3.7)$$

Backward direction

$$h_t = \text{LSTM}(F_{\text{multi},t}, h_{t+1}) \quad (3.8)$$

The bidirectional hidden state at each time step is obtained by concatenating the forward and backward hidden states:

$$h_t = [\vec{h}_t \parallel h_t], \quad h_t \in \mathbb{R}^{2H} \quad (3.9)$$

where H denotes the number of hidden units in each directional LSTM. The internal dynamics of each LSTM unit are governed by the standard gating mechanism:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (3.10)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3.11)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (3.12)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (3.13)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (3.14)$$

$$h_t = o_t \odot \tanh(C_t) \quad (3.15)$$

where f_t , i_t , and o_t denote the forget gate, input gate, and output gate, respectively; C_t is the cell state responsible for carrying long-term memory across time steps; σ is the sigmoid activation; and $\odot$ represents element-wise multiplication.

The forget gate, f_t , decides how much of the previous cell state sticks around, giving the model control to toss out old info that doesn't matter for the current task. The input gate, i_t , handles how much of the new candidate memory, $\tilde{C}_t$, gets added to the cell state. Then the output gate, o_t , filters what part of the cell state gets revealed as the hidden state at that moment. Thanks to this gating setup, LSTMs don't struggle with vanishing gradients like standard RNNs do—they're great at holding onto information over long sequences.

In this model, I stacked two BiLSTM layers. The first BiLSTM processes features from the multi-scale convolutions, and its output—fine-tuned through a gated residual skip connection—feeds into the second BiLSTM. This way, the bottom layer picks up on short-term patterns, while the top layer catches the bigger-picture dynamics. I also included dropout between the stacked layers to keep the model from overfitting and to help it generalize better.

Chapter 4 - Non-Wearable Sensor Dataset

4.1 Dataset Overview

This study used a non-wearable sensor dataset collected with the Microsoft Kinect v2. Unlike traditional motion capture systems that need you to suit up in tight-fitting markers, the Kinect v2 works without anything attached to your body. It's a depth-sensing device that uses time-of-flight technology—it basically shoots out infrared light, measures how long it takes to bounce back, and puts the data into a detailed depth map of the scene in real time. Then, it runs a pose estimation algorithm to figure out the precise three-dimensional coordinates of 25 anatomical joints all across the body. The sensor grabs both RGB color and depth images at once, letting it capture subtle human motion as it actually happens, without getting in the way or making people feel self-conscious. That's a huge advantage, especially in homes, clinics, or assisted living spaces, where you want people moving naturally and not worrying about what they're wearing for the sake of technology.

To build a dataset that actually reflects real-world diversity, the researchers recruited 30 volunteers. They didn't just round up a bunch of college students—they looked for people of different ages, including both young adults and older folks, and made sure to include a good mix of genders, heights, and body shapes. This mix was intentional. Human movement patterns can vary a lot across individuals; by capturing a wide range, the team made sure that models trained on this dataset won't get thrown off by "unusual" bodies or movements. The goal: help machine learning algorithms see patterns that hold up across all sorts of people, not just a narrow slice of the population.

Each volunteer was asked to perform seven different activities: bending, standing, walking, sitting, jumping, squatting, and running. The point was to cover a big cross-section of how people move in daily life. Some of these actions, like standing or sitting, are pretty basic and slow-paced. Others, like jumping or running, involve faster, more complex motion and put the skeletal tracking to the test in a different way. Together, these activities create a well-rounded dataset, capturing everything from everyday movements to more physically demanding actions. This range helps ensure that any recognition system built using this data won't just work in the

lab, but will also handle the unpredictable, varied movement found in real-world scenarios.

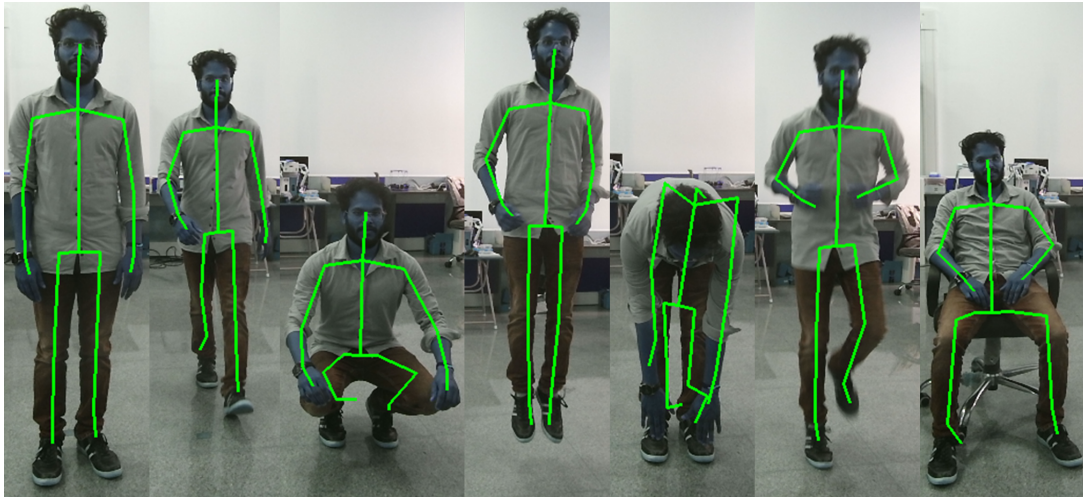


Figure 4.1: Volunteer performing Activity for Skeletal Dataset Collection

These activities were selected to represent a broad spectrum of human movement categories, as summarized below:

- **Standing:** A stationary upright posture with minimal motion, serving as the baseline static class.
- **Walking and Running:** Cyclic locomotion activities distinguished by gait frequency and the presence of a flight phase.
- **Sitting:** A postural transition from upright to seated position followed by a sustained seated posture.
- **Bending:** Forward flexion of the torso, commonly encountered in daily tasks such as object retrieval.
- **Squatting:** Coordinated flexion of the hips, knees, and ankles while maintaining an upright torso orientation.
- **Jumping:** A ballistic movement with distinct preparatory, flight, and landing phases.

Collectively, these seven classes capture the essential movement primitives composing more complex activity sequences encountered in daily life.

4.2 Data Collection Protocol

We set up data collection in a carefully controlled indoor space, making sure there was more than enough room for each person to move around freely, no matter which activity they were about to perform. To capture everything, we used a Kinect v2 sensor fixed tightly to a sturdy

tripod. We placed it at about 1.2 meters off the ground, and set it between 2.5 and 3.5 meters away from where the action would take place. This distance gave the sensor full coverage and made sure participants stayed inside its depth-sensing zone at all times. Before we even started recording, we ran through sensor calibration and double-checked the whole area—no objects or odd surfaces that might mess with the body tracking, just clean space for accurate skeletal data.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
timestamp	SpineBase	SpineBase	SpineBase	SpineBase	SpineMid	SpineMid	SpineMid	Neck_x	Neck_y	Neck_z	Head_x	Head_y	Head_z	ShoulderL	ShoulderL	ShoulderL	ElbowLeft	ElbowLeft	ElbowLeft	WristLeft	WristLeft	WristLeft	
0.001411	5	-0.02176	-0.30516	2.630938	-0.0137	-0.07557	2.391607	-0.00614	0.15387	2.144708	-0.0026	0.236688	2.056595	-0.17219	0.087046	2.216316	-0.23468	-0.12238	2.371286	-0.235	-0.3456	2.416762	
0.032645	5	-0.02084	-0.29716	2.623193	-0.01323	-0.06201	2.389219	-0.00595	0.173328	2.144095	-0.00256	0.259951	2.06092	-0.17135	0.101215	2.214744	-0.23618	-0.11539	2.366459	-0.23177	-0.33837	2.412957	
0.066819	5	-0.02096	-0.28964	2.61602	-0.01426	-0.04647	2.387104	-0.00815	0.197917	2.148689	-0.00502	0.294808	2.068954	-0.17316	0.115738	2.217129	-0.23588	-0.10349	2.359873	-0.23098	-0.32586	2.40884	
0.100852	5	-0.02179	-0.28112	2.605728	-0.01622	-0.03028	2.381581	-0.01115	0.219524	2.146884	-0.00623	0.316395	2.068332	-0.17545	0.129886	2.216129	-0.23634	-0.09626	2.358284	-0.23277	-0.31986	2.402881	
0.132388	5	-0.02237	-0.27356	2.598287	-0.01796	-0.0171	2.379832	-0.01428	0.238926	2.151202	-0.00768	0.337239	2.073318	-0.17815	0.145999	2.217822	-0.23668	-0.08827	2.358818	-0.2333	-0.31309	2.398679	
0.165583	5	-0.02257	-0.26388	2.587375	-0.01918	-0.00237	2.376657	-0.01634	0.258035	2.154652	-0.01098	0.362834	2.079329	-0.18213	0.163686	2.21857	-0.2377	-0.08136	2.359119	-0.23643	-0.3074	2.393727	
0.199905	5	-0.02202	-0.25664	2.579898	-0.02001	0.010268	2.376112	-0.01887	0.277812	2.162845	-0.01332	0.384308	2.08911	-0.18254	0.166316	2.219373	-0.23968	-0.06709	2.358351	-0.23835	-0.29411	2.388505	
0.232735	5	-0.02359	-0.24365	2.566056	-0.02259	0.029217	2.373453	-0.02228	0.301581	2.170027	-0.01627	0.412719	2.108801	-0.189	0.201234	2.227303	-0.24326	-0.04963	2.357954	-0.24219	-0.27908	2.38166	
0.265948	5	-0.02343	-0.23232	2.552305	-0.02271	0.044964	2.371071	-0.02219	0.322277	2.178119	-0.01876	0.436987	2.108647	-0.19114	0.21339	2.229852	-0.24391	-0.03885	2.359855	-0.2442	-0.26791	2.378265	
0.299904	5	-0.02303	-0.22083	2.53779	-0.02328	0.061254	2.36883	-0.02377	0.344136	2.188215	-0.02142	0.459488	2.119787	-0.19342	0.230426	2.235002	-0.24671	-0.02428	2.359641	-0.24761	-0.2542	2.371886	
0.333452	5	-0.0228	-0.21387	2.525687	-0.02374	0.066016	2.371549	-0.02593	0.350302	2.218116	-0.02299	0.480511	2.13303	-0.19662	0.243206	2.240956	-0.25272	-0.006537	2.354991	-0.25073	-0.22981	2.363255	
0.365307	5	-0.02373	-0.20973	2.517939	-0.02479	0.072814	2.373666	-0.02679	0.356691	2.225102	-0.02503	0.494365	2.144209	-0.20067	0.253583	2.247549	-0.25496	-0.0131	2.354534	-0.25407	-0.22252	2.357101	
0.401815	5	-0.02467	-0.20664	2.511433	-0.02501	0.078707	2.374787	-0.02576	0.364738	2.231394	-0.02644	0.504211	2.152454	-0.20307	0.260158	2.252637	-0.25675	-0.017085	2.353725	-0.25599	-0.21965	2.353476	
0.43234	5	-0.02533	-0.20554	2.506952	-0.02524	0.081996	2.375071	-0.02544	0.369515	2.235357	-0.02838	0.511182	2.15779	-0.2049	0.264156	2.256399	-0.2591	-0.020322	2.352613	-0.26028	-0.2142	2.344007	
0.464874	5	-0.02595	-0.20525	2.50501	-0.02548	0.083744	2.37566	-0.02518	0.372084	2.238286	-0.02923	0.513226	2.160207	-0.20581	0.265723	2.258623	-0.2602	-0.021165	2.351767	-0.26262	-0.21423	2.341312	
0.499911	5	-0.02596	-0.20527	2.504656	-0.0255	0.084289	2.376176	-0.02515	0.372424	2.239491	-0.02945	0.51292	2.160825	-0.20604	0.266133	2.259273	-0.26145	-0.021628	2.349561	-0.26614	-0.21592	2.334515	
0.532602	5	-0.02599	-0.20556	2.504738	-0.02547	0.084084	2.376579	-0.025	0.37148	2.239849	-0.02963	0.510995	2.160334	-0.20627	0.26552	2.259578	-0.26262	-0.020624	2.346619	-0.2679	-0.21871	2.32396	
0.564872	5	-0.02629	-0.20709	2.507241	-0.02551	0.08171	2.377863	-0.02455	0.363909	2.238439	-0.0301	0.497875	2.154349	-0.20575	0.262262	2.259164	-0.26334	-0.012698	2.341205	-0.27014	-0.22511	2.324046	
0.600919	5	-0.02689	-0.21333	2.515783	-0.02628	0.07127	2.380955	-0.02343	0.342669	2.233199	-0.02747	0.489919	2.144253	-0.20409	0.251303	2.258809	-0.26355	-0.002161	2.334866	-0.27164	-0.23193	2.321027	
0.63243	5	-0.03132	-0.22607	2.531742	-0.02807	0.064213	2.378061	-0.02284	0.333992	2.19953	-0.02567	0.441672	2.119876	-0.20116	0.230156	2.249455	-0.26451	-0.0182	2.328084	-0.27176	-0.24033	2.319898	
0.666214	5	-0.03307	-0.23902	2.545754	-0.02756	0.043273	2.375745	-0.02202	0.312382	2.185546	-0.02414	0.409906	2.105477	-0.19885	0.208997	2.240844	-0.26411	-0.03986	2.323844	-0.27207	-0.27041	2.319898	
0.701723	5	-0.03421	-0.25598	2.564825	-0.02748	0.01858	2.378382	-0.02047	0.278782	2.175563	-0.02027	0.370911	2.093779	-0.19488	0.180055	2.23618	-0.2663	-0.05317	2.319327	-0.27148	-0.28503	2.325508	
0.734526	5	-0.03559	-0.27719	2.58375	-0.02575	0.01106	2.38096	-0.01519	0.243322	2.16233	-0.01733	0.332763	2.079619	-0.19377	0.144632	2.233371	-0.26458	-0.08205	2.321681	-0.26942	-0.31299	2.334623	
0.766618	5	-0.03657	-0.29191	2.597469	-0.02451	-0.03988	2.386725	-0.01151	0.20128	2.15897	-0.01392	0.281572	2.073799	-0.19212	0.12155	2.229041	-0.26305	-0.10362	2.327386	-0.26629	-0.3327	2.343043	
0.801197	5	-0.03981	-0.31283	2.619394	-0.02587	-0.07238	2.394322	-0.01174	0.157202	2.154009	-0.01447	0.236904	2.067708	-0.18712	0.096442	2.224824	-0.26353	-0.12457	2.322901	-0.26277	-0.35555	2.353096	

Figure 4.2: Skeletal Dataset Sample

Each participant got both a clear verbal rundown and visual demonstrations of all seven activities. Everyone was told to move as naturally as possible—do it their own way, with no forced, cookie-cutter choreography. We wanted the data to reflect real movement patterns, the kind of variability you actually see when different people perform the same activity. That’s a major challenge for any serious activity recognition system, and one we wanted our dataset to capture head-on. For activities that naturally repeat (like walking or running), participants did several cycles back-to-back. For single-step actions like jumping or squatting, they repeated the movement a few times with brief rests in between. When it came to transitions—think sitting down, bending over, and then returning to the original stance—each participant moved through the full motion from start to finish.

We recorded the skeletal joint data at about 30 frames per second. Each frame logged the 3D coordinates (x for left-right, y for up-down, z for depth) of 25 key joints, mapped in the Kinect’s camera coordinate system. These numbers, accurate down to the meter, came as floating point values relative to the sensor’s origin. The 25 joints spanned the body from head to toe—spine, neck, both arms, both legs, hands, feet, and all the main connection points in between—mirroring the complete human movement chain. The list covered everything: SpineBase, SpineMid, Neck, Head, ShoulderLeft, ElbowLeft, WristLeft, HandLeft, ShoulderRight, ElbowRight, WristRight, HandRight, HipLeft, KneeLeft, AnkleLeft, FootLeft, HipRight, KneeRight, AnkleRight, FootRight, SpineShoulder, HandTipLeft, ThumbLeft, HandTipRight, and ThumbRight.

Once we had all the motion data, we organized it into clean, easy-to-handle CSV files. Every row captured a snapshot—one frame in time. The columns started with an exact timestamp, marking how many seconds had passed since the beginning of the session. The next field

identified which body the Kinect was tracking, handy in case more than one person showed up in the frame. The main bulk of each row—75 columns—held the xyz coordinates for those 25 tracked joints (three numbers each, always in the same order). Finally, the last two columns tagged the participant and precisely labeled which activity they were performing in each frame. By keeping the structure this organized, we set things up perfectly for seamless loading into a deep learning pipeline, making future model training and analysis as frictionless as possible.

4.3 Dataset Characteristics and Preprocessing

The dataset covers several hours of skeletal recordings from a diverse group of participants, capturing a wide range of activity types. Each activity and participant contributes a different amount of data—for example, when someone stands still, the system records longer stretches of almost unchanged skeleton positions, making those sequences pretty stable. In contrast, dynamic activities like jumping produce much shorter sequences, and the skeletal data jumps around quite a bit since people’s joints move rapidly and unpredictably.

Kinect isn’t perfect, especially at the edges of the body. You see more noise at the hands and feet, mostly because depth sensing struggles when those joints get blocked or are far from the camera. To handle that and keep things fair, a comprehensive preprocessing pipeline is applied to ensure data quality and model robustness.

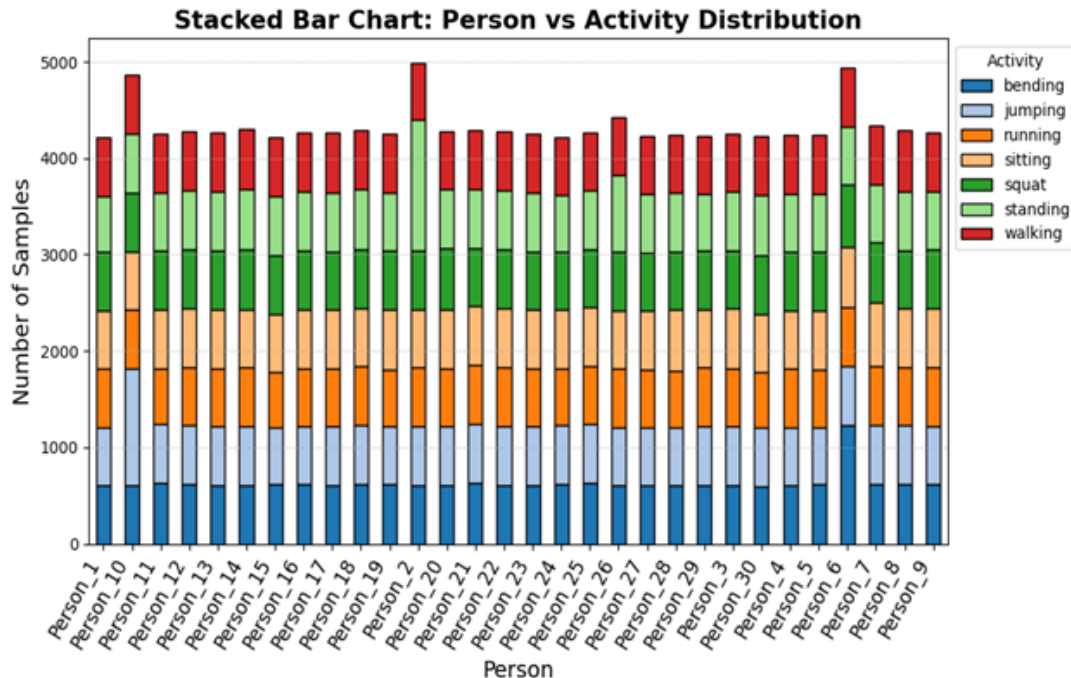


Figure 4.3: Person vs Activity Distribution on Skeletal Dataset

4.3.1 Data Loading and Initial Processing

The raw skeletal data is organized in a hierarchical structure with separate directories for each participant (`Person_1`, `Person_2`, etc.), where each directory contains CSV files named after the corresponding activities (`walking.csv`, `running.csv`, etc.). Each CSV file contains temporal sequences with columns for timestamp, body ID, and 3D coordinates (x, y, z) for each skeletal joint.

Let $\mathbf{R}_i \in \mathbb{R}^{T_i \times (2+3J)}$ represent the raw data for recording i , where T_i is the number of frames, J is the number of skeletal joints, and the first 2 columns contain timestamp and body ID. The initial preprocessing extracts only the joint coordinate features:

$$\mathbf{X}_i^{\text{raw}} = \mathbf{R}_i[:, 2:] \in \mathbb{R}^{T_i \times 3J} \quad (4.1)$$

Missing or corrupted values—which occasionally occur due to occlusion or sensor limitations—are handled through a multi-stage imputation strategy. For a feature vector $\mathbf{x}_f$ with missing values at indices $\mathcal{M}$:

$$x_f[t] = \begin{cases} x_f[t-1] & \text{if } t \in \mathcal{M} \text{ and } t-1 \notin \mathcal{M} \text{ (forward fill)} \\ x_f[t+1] & \text{if } t \in \mathcal{M} \text{ and } t+1 \in \mathcal{M} \text{ (backward fill)} \\ 0 & \text{if } t \in \mathcal{M} \text{ and all neighbors are missing} \\ x_f[t] & \text{otherwise} \end{cases} \quad (4.2)$$

This ensures complete, clean numeric arrays for each recording.

4.3.2 Temporal Segmentation via Sliding Windows

For preparing the model’s input, the continuous recordings are chopped up into fixed-length pieces with a sliding window approach. Each window holds $W = 64$ frames—which works out to approximately two seconds of movement since the Kinect tracks at 30 frames per second. The window slides ahead by $S = 16$ frames at a time, creating a 75% overlap between consecutive segments.

For a sequence $\mathbf{X}_i^{\text{raw}} \in \mathbb{R}^{T_i \times F}$ where $F = 3J$ is the number of features, the sliding window operation produces:

$$\mathbf{W}_i^{(k)} = \mathbf{X}_i^{\text{raw}}[kS : kS + W, :] \in \mathbb{R}^{W \times F} \quad (4.3)$$

for $k = 0, 1, 2, \dots, K_i - 1$ where $K_i = \lfloor \frac{T_i - W}{S} \rfloor + 1$ is the total number of windows extracted from recording i .

For sequences shorter than W frames (i.e., $T_i < W$), zero-padding is applied:

$$\mathbf{X}_i^{\text{padded}} = \begin{bmatrix} \mathbf{X}_i^{\text{raw}} \\ \mathbf{0}_{(W-T_i) \times F} \end{bmatrix} \in \mathbb{R}^{W \times F} \quad (4.4)$$

This windowing strategy serves dual purposes: it provides better temporal coverage of transitions between motion phases and significantly augments the training set size. Each window $\mathbf{W}_i^{(k)}$ is assigned the activity label y_i corresponding to recording i . Windows that span activity transitions are discarded to prevent label ambiguity.

4.3.3 Feature Normalization

Every skeletal coordinate—across every joint and every axis—undergoes standardization using z-score normalization. Let $\mathcal{D} = \{\mathbf{W}_i^{(k)}\}$ represent the collection of all windowed samples. The normalization is applied per-feature across all timesteps by first reshaping the data from (N, W, F) to $(N \cdot W, F)$ where N is the total number of windows, computing mean μ_f and standard deviation σ_f for each feature dimension f :

$$\mu_f = \frac{1}{N \cdot W} \sum_{i=1}^N \sum_{t=1}^W x_{i,t,f} \quad (4.5)$$

$$\sigma_f = \sqrt{\frac{1}{N \cdot W} \sum_{i=1}^N \sum_{t=1}^W (x_{i,t,f} - \mu_f)^2} \quad (4.6)$$

The standardized features are then computed as:

$$\tilde{x}_{i,t,f} = \frac{x_{i,t,f} - \mu_f}{\sigma_f + \epsilon} \quad (4.7)$$

where $\epsilon = 10^{-8}$ is a small constant to prevent division by zero. This ensures that no matter if a certain body part moves a lot or only a little, it won't overpower the training—every feature carries similar weight with zero mean and unit variance.

To simulate real-world sensor imperfections and improve model robustness, Gaussian noise is injected into the normalized features:

$$\hat{x}_{i,t,f} = \tilde{x}_{i,t,f} + \mathcal{N}(0, \sigma_{\text{noise}}^2) \quad (4.8)$$

where $\sigma_{\text{noise}} = 0.05$ and $\mathcal{N}(0, \sigma_{\text{noise}}^2)$ denotes a Gaussian distribution with zero mean and variance σ_{noise}^2 . This data augmentation technique acts as a regularizer, helping the model generalize better to noisy deployment conditions while preventing overfitting to pristine training samples.

4.3.4 Label Encoding and Data Splitting

Activity labels, originally stored as strings $\mathcal{L} = \{l_1, l_2, \dots, l_C\}$ where C is the number of activity classes (e.g., "walking", "running"), are converted to integer encodings:

$$y_i = \text{encode}(l_i) \in \{0, 1, 2, \dots, C - 1\} \quad (4.9)$$

These integer labels are subsequently transformed into one-hot encoded vectors suitable for multi-class classification:

$$\mathbf{y}_i^{\text{one-hot}} = [y_{i,0}, y_{i,1}, \dots, y_{i,C-1}]^T \text{ where } y_{i,c} = \begin{cases} 1 & \text{if } c = y_i \\ 0 & \text{otherwise} \end{cases} \quad (4.10)$$

The finished dataset $\mathcal{D} = \{(\hat{\mathbf{X}}_i, \mathbf{y}_i^{\text{one-hot}})\}_{i=1}^N$ is then split into training and test sets using an 80-20 stratified partition:

$$\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}} = \text{split}(\mathcal{D}, \text{ratio} = 0.8, \text{stratify} = \mathbf{y}) \quad (4.11)$$

The stratification ensures that the class distribution is preserved in both sets:

$$\frac{|\{i : y_i = c, i \in \mathcal{D}_{\text{train}}\}|}{|\mathcal{D}_{\text{train}}|} \approx \frac{|\{i : y_i = c, i \in \mathcal{D}_{\text{test}}\}|}{|\mathcal{D}_{\text{test}}|} \approx \frac{|\{i : y_i = c\}|}{N} \quad (4.12)$$

for all classes $c \in \{0, 1, \dots, C - 1\}$. This is vital—otherwise, if some activities happen more often, the model would just learn those and ignore the rare ones, dooming it to poor generalization. By keeping the distribution balanced, the system’s evaluation reflects real performance across the board, not just on the most frequent classes.

An alternative person-level splitting strategy is also available, where train and test sets are divided based on participant identity rather than randomly shuffled windows:

$$\mathcal{P}_{\text{train}}, \mathcal{P}_{\text{test}} = \text{split}(\mathcal{P}, \text{ratio} = 0.8) \quad (4.13)$$

$$\mathcal{D}_{\text{train}} = \{(\mathbf{X}_i, \mathbf{y}_i) : p_i \in \mathcal{P}_{\text{train}}\}, \quad \mathcal{D}_{\text{test}} = \{(\mathbf{X}_i, \mathbf{y}_i) : p_i \in \mathcal{P}_{\text{test}}\} \quad (4.14)$$

where p_i denotes the participant ID for sample i . This ensures zero data leakage between sets and provides a more stringent test of generalization to unseen individuals.

4.3.5 Multi-Scale Temporal Downsampling

Beyond standard preprocessing, the model architecture itself incorporates multi-scale temporal processing. The normalized input sequences are processed at three different temporal resolutions simultaneously through strided temporal sampling:

$$\mathbf{X}^{(0)} = \hat{\mathbf{X}} \in \mathbb{R}^{W \times F} \quad (\text{full resolution}) \quad (4.15)$$

$$\mathbf{X}^{(1)} = \hat{\mathbf{X}}[:, 2:, :] \in \mathbb{R}^{W/2 \times F} \quad (\text{half resolution, stride 2}) \quad (4.16)$$

$$\mathbf{X}^{(2)} = \hat{\mathbf{X}}[:, 4:, :] \in \mathbb{R}^{W/4 \times F} \quad (\text{quarter resolution, stride 4}) \quad (4.17)$$

where the notation $[:, \tau, :]$ indicates selecting every τ -th frame along the temporal dimension. Mathematically, for scale s with stride τ_s :

$$\mathbf{X}^{(s)}[t, :] = \hat{\mathbf{X}}[t \cdot \tau_s, :], \quad t = 0, 1, \dots, \lfloor W/\tau_s \rfloor - 1 \quad (4.18)$$

This multi-scale approach enables the network to capture both fine-grained motion details and coarse-grained temporal patterns, allowing it to learn hierarchical representations of human activities at different temporal granularities.

Table 4.1: Preprocessing Pipeline Summary

Step	Operation	Parameters
Data Loading	Drop timestamp & body ID columns	-
Missing Value Handling	Forward/backward fill, zero imputation	Eq. (2)
Sliding Window	Fixed-length segmentation	$W = 64, S = 16$ (Eq. 3-4)
Normalization	Z-score standardization	Per-feature (Eq. 5-7)
Label Encoding	String to integer mapping	Eq. (9)
One-Hot Encoding	Integer to categorical vector	Eq. (10)
Train/Test Split	Stratified random split	80%/20% (Eq. 11-12)
Multi-Scale Sampling	Temporal downsampling	Strides: $\{1, 2, 4\}$ (Eq. 14-16)

Altogether, with the wide range of participants, natural differences in how people perform each activity, and this careful, robust preprocessing routine, the resulting dataset offers a truly challenging and realistic testbed. The mathematical rigor of the preprocessing pipeline ensures reproducibility and provides a solid foundation for training sophisticated models like the MS-GRS-BiLSTM, which aim to handle the complexities and quirks of human motion in the wild.

4.4 Skeletal Dataset Results

This study takes a close look at deep learning methods for human activity recognition, working across two very different sensing types. We evaluated four sequential models—LSTM, BiLSTM, GRS-BiLSTM, and our own MS-GRS-BiLSTM—on a wide range of datasets. These included both popular benchmarks and new data we collected ourselves, covering non-wearable skeletal sensors and wearable mobile inertial sensors.

The results are clear: MS-GRS-BiLSTM stands out, delivering the best recognition performance on every metric, across all datasets and sensing types. These findings make a strong case for our approach. Combining multi-scale temporal decomposition, gated residual skip connec-

tions, and bidirectional recurrent processing really does strengthen human activity recognition. Each piece adds value, and together, they create a model that’s more robust and accurate.

We ran non-wearable sensor experiments on two skeletal datasets. The first, our own, came from 30 volunteers doing seven types of activities, and the second was the public Kinect Activity Recognition Dataset (KARD).

On our dataset, the results speak for themselves. The basic LSTM model reached a test accuracy of 76.19% (F1: 0.7527), but it struggled since it only looks forward in time. Introducing bidirectionality made a big difference—BiLSTM jumped up to 88.10% (F1: 0.8700), making it clear that having context from both past and future frames matters. Taking it a step further, the GRS-BiLSTM pushed accuracy to 93.33% (F1: 0.9300), thanks to gated residual connections that help preserve all the subtle skeletal movements. But the strongest performer by far was our MS-GRS-BiLSTM. It not only topped out with 98.35% accuracy (precision: 0.9836, recall: 0.9833, F1: 0.9834)—about 5 percentage points better than GRS-BiLSTM and more than 22 points better than the vanilla LSTM—but it was also the most efficient. Its model size was just 0.982 MB, and it processed each sample in 1.857 ms, showing that you don’t have to trade speed or size for top-tier results.

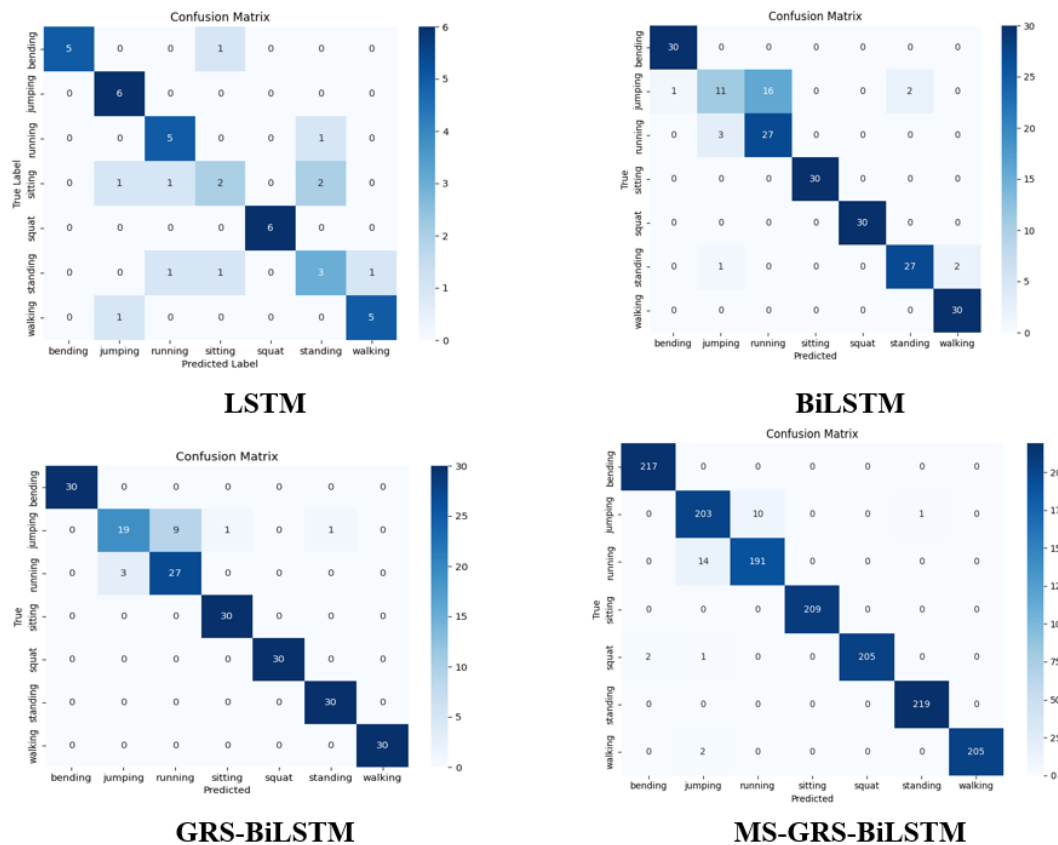


Figure 4.4: Confusion Matrix on Skeletal Dataset

Table 4.2: Classwise comparison of Accuracies over models on Skeletal Dataset

Activities / Models	LSTM	BiLSTM	GRS-BiLSTM	Proposed Model
Bending	0.9091	0.9800	1.0000	0.9977
Jumping	0.8571	0.4900	0.7300	0.9469
Running	0.7692	0.7400	0.8200	0.9467
Sitting	0.4000	1.0000	0.9800	1.0000
Squat	1.0000	1.0000	1.0000	0.9976
Standing	0.5000	0.9200	0.9800	0.9977
Walking	0.8333	0.9700	1.0000	0.9971

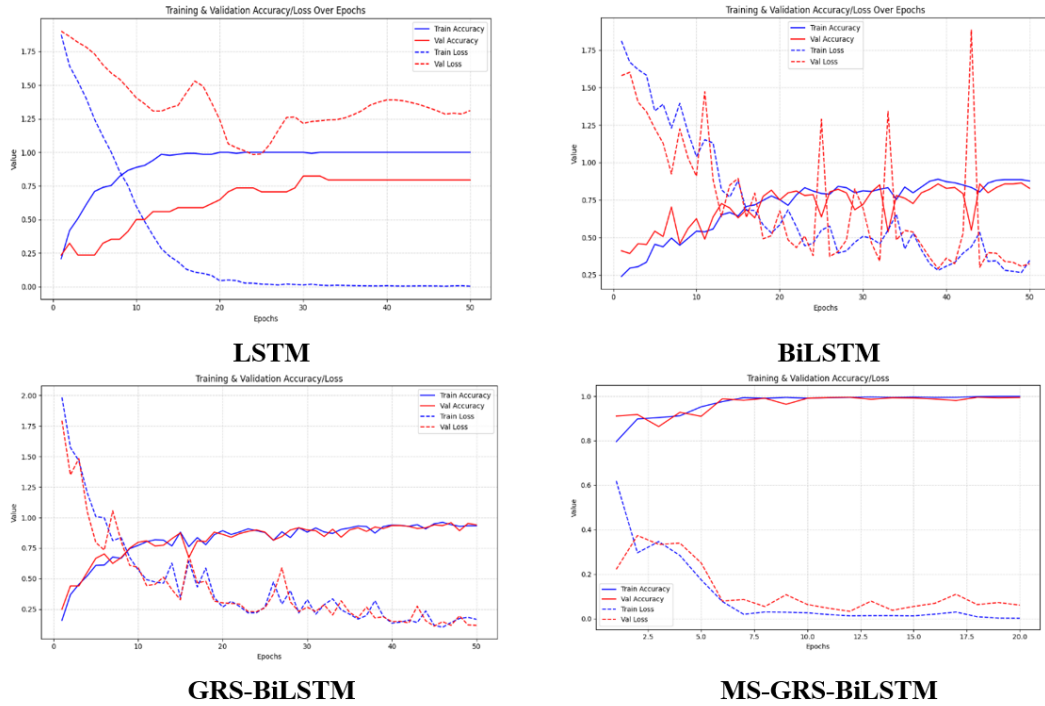


Figure 4.5: Accuracy/Loss curve over training on Skeletal Dataset

Table 4.3: Metrics comparison over models on Skeletal Dataset

Model	Test Loss	Test Accuracy	Precision	Recall	F1-Score	Model Size	Avg Inference Time per Sample
LSTM	1.3108	76.19%	0.7568	0.7619	0.7527	1.86 MB	31.01 ms
BiLSTM	0.3456	88.10%	0.8900	0.8800	0.8700	1.43 MB	27.03 ms
GRS-BiLSTM	0.1671	93.33%	0.9400	0.9300	0.9300	22.56 MB	31.86 ms
MS-GRS-BiLSTM	0.0516	98.35%	0.9836	0.9833	0.9834	0.982 MB	1.857 ms

4.5 KARD Dataset Results

The trend was nearly identical on the KARD benchmark. LSTM hit 77.78% (F1: 0.7587), BiLSTM 87.65% (F1: 0.8740), and GRS-BiLSTM 92.59% (F1: 0.9240). Again, MS-GRS-BiLSTM led the pack, scoring 98.77% accuracy with precision at 0.9907, recall at 0.9861, F1-score at 0.9870, and a lightning-fast inference time of just 0.020 ms per sample. Seeing such consistent performance across two very different skeletal datasets—our own and KARD—gives strong evidence that this architecture can generalize well, regardless of the data source or the range of activities.

Table 4.4: Metrics comparison over models on KARD Dataset

Model	Test Loss	Test Accuracy	Precision	Recall	F1-Score	Model Size	Inference Time per Sample
LSTM	0.9281	77.78%	0.7792	0.7750	0.7587	0.04 MB	0.015 ms
BiLSTM	0.5001	87.65%	0.8896	0.8750	0.8740	0.09 MB	0.077 ms
GRS-BiLSTM	0.3790	92.59%	0.9697	0.9222	0.9240	0.29 MB	0.037 ms
MS-GRS-BiLSTM	0.0573	98.77%	0.9907	0.9861	0.9870	0.476 MB	0.020 ms

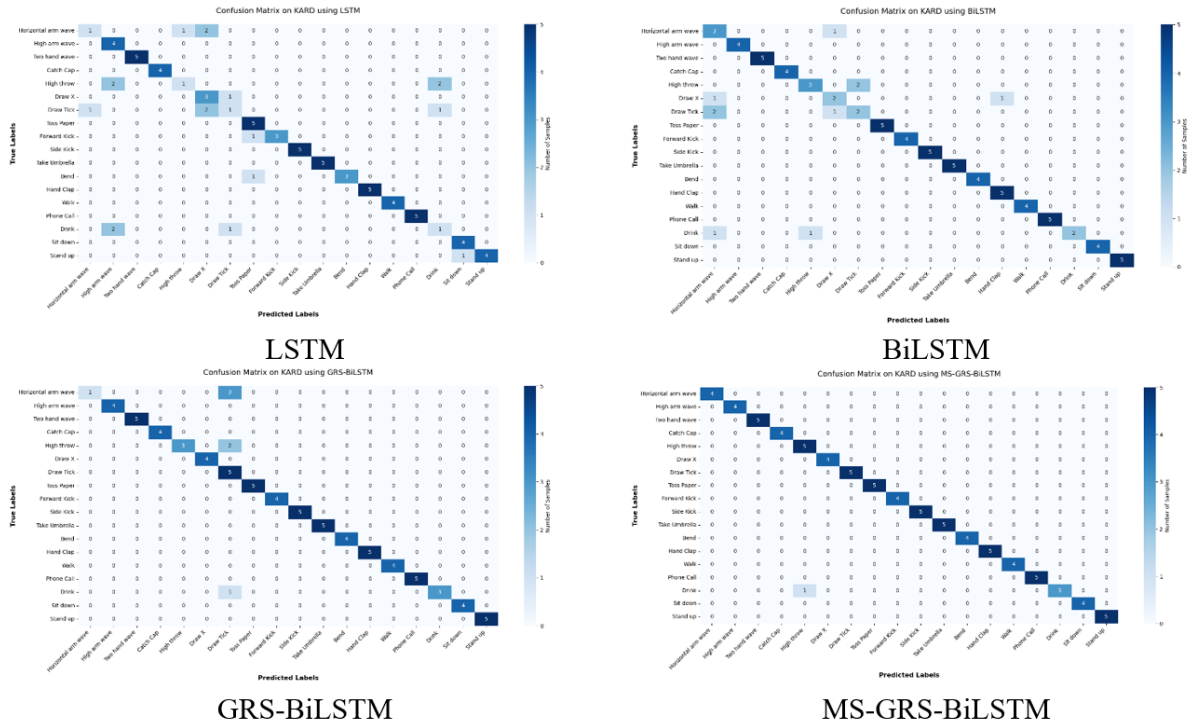
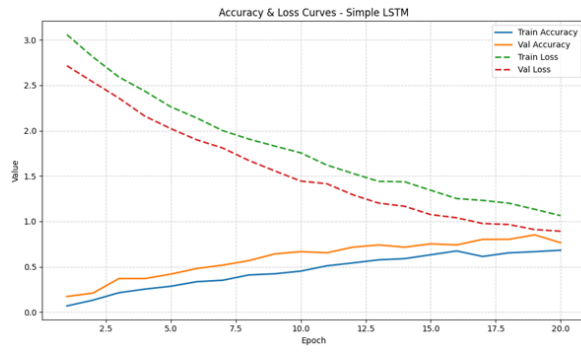
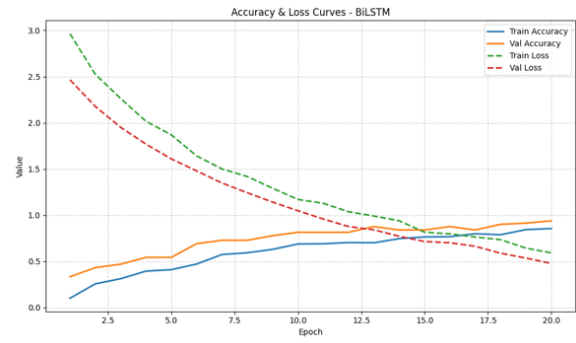


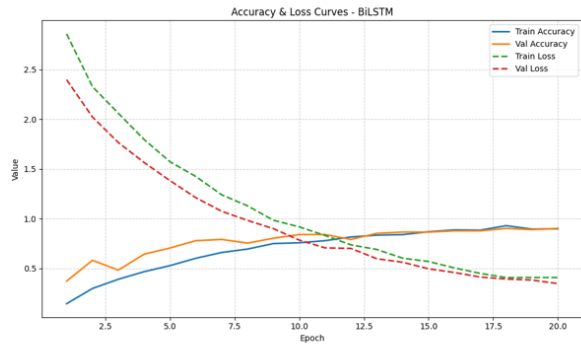
Figure 4.6: Confusion Matrix on KARD Dataset



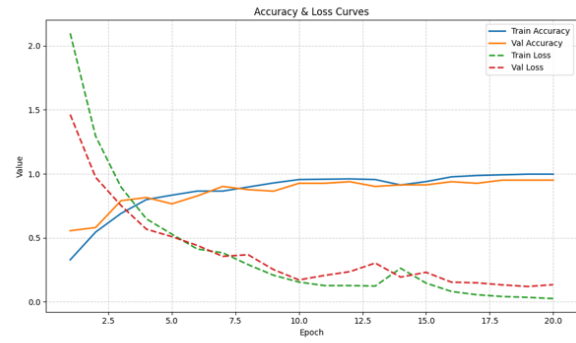
LSTM



BiLSTM



GRS-BiLSTM



MS-GRS-BiLSTM

Figure 4.7: Accuracy/Loss over epoch on KARD Dataset

Chapter 5 - Mobile Sensor Dataset

5.1 Dataset Overview

We built this wearable sensor dataset by tapping into the triaxial accelerometer and gyroscope packed inside a modern Android smartphone—tiny MEMS (micro-electromechanical systems) marvels that capture the fine details of how people move. These sensors, working in three directions, constantly track every twist, turn, and step you take, all while staying out of the way and not requiring any extra gear or installation. Gathering the data was pretty straightforward: we used an app called Sensor Logger, which let us pull raw, unfiltered motion data complete with millisecond-accurate timestamps. Everything got exported as clean CSV files for easy handling later on.

To keep things fair and consistent, we made sure everyone wore the phone in the same spot for every recording. We placed it right on the chest using Valkyrie tape, not just because it's a convenient place, but because the chest sits close to the body's center of mass. It's right where the real action happens, especially during whole-body movements, and it keeps the phone stable and aligned even during jumping or sudden shifts—basically, the ideal spot for minimizing sensor wobble and confusing data.

We didn't just use a handful of similar volunteers, either. The dataset includes data from 30 different people—covering a good mix of ages, genders, body types, heights, and fitness levels. This variety isn't just for show; it means any machine learning model trained on this data can handle the messiness and unpredictability of real-world users, not just ideal lab conditions.

Each person went through seven types of activity: bending, standing, squatting, running, jumping, walking, and sitting. This range captures everything from still poses to frantic action, plus those in-between transitions that often trip up activity recognition systems. By matching this activity list to our skeletal dataset, we set the stage for a direct, apples-to-apples comparison between how well wearables and camera-based systems can tell what someone's doing. The end result isn't just a raw collection of numbers—it's a rich, realistic foundation for building smarter, more adaptable human activity models.



Figure 5.1: Volunteer performing Activity for Mobile Dataset Collection

5.2 Data Collection Protocol

We set up data collection in both indoor and outdoor spaces, making sure there was enough room for people to move around during each activity. It wasn't just about keeping things organized—the extra space ensured that no movement was limited, which helps when you want truly natural motion data. The team used a smartphone to record everything, locked in at a steady 50 Hz sampling rate. At this frequency, the phone didn't miss out on any quick shifts in movement; it's high enough to catch all the little details you want when tracking someone's motions.

To make sure the phone didn't move around and mess up the readings, we strapped it to each participant's chest using Valkyrie tape. The Sensor Logger app did the heavy lifting, recording all the necessary motion data while everyone went through the activities. Before we started hitting 'record,' though, we always gave a clear run-through. People got both a spoken explanation and a live demo for all seven activities—no guessing, no confusion.

But here's the key part: participants weren't told to match some rigid example or move in a specific way. Instead, we asked them to go through the motions in their own natural style. This meant we captured not just the base activity, but all those little differences from person to person—real-world variability that you just can't fake.

For cyclic activities like walking or running, participants just kept going for 30 to 60 seconds, letting us gather enough cycles for solid analysis. When it came to actions like jumping or squatting, which are more discrete, people did several reps with quick pauses in between. Then, for transitional moves like sitting or bending, we had them complete entire sequences a few times to capture how people naturally shift between positions.

Throughout all this, the Sensor Logger app recorded two main sets of data: linear acceleration and angular velocity. Accelerometer readings came in three channels (acc_x , acc_y , acc_z) measured in m/s^2 , capturing every forward, sideways, and up-and-down motion. Gyroscope data also flowed through three channels (gyro_x , gyro_y , gyro_z), tracking rotational changes

A	B	C	D	E	F	G	H	I
time	acc_x	acc_y	acc_z	gyro_x	gyro_y	gyro_z	activity	person_id
1.76E+18	0.150297	0.475871	1.692409	-0.16618	-0.22051	-0.13156	bending	Person_1
1.76E+18	0.440364	-0.78892	0.744152	-0.16618	-0.22051	-0.13156	bending	Person_1
1.76E+18	-0.65777	-0.52659	0.292423	1.234109	-0.51612	-0.61199	bending	Person_1
1.76E+18	0.061683	0.786837	0.664982	-0.06711	-2.16675	-0.29135	bending	Person_1
1.76E+18	-0.9288	1.105371	-0.73457	-0.33556	-2.19817	-0.17737	bending	Person_1
1.76E+18	-1.32234	0.400625	-3.69007	0.142745	-2.1513	-0.5076	bending	Person_1
1.76E+18	-0.36768	0.26491	-4.08284	-1.53345	-2.05916	0.305198	bending	Person_1
1.76E+18	0.527967	-0.04098	-2.11172	-1.26926	-2.39205	-0.11185	bending	Person_1
1.76E+18	0.838848	0.666696	1.778725	0.39468	-2.76436	-0.457	bending	Person_1
1.76E+18	0.860426	-0.01679	-1.0513	1.279383	-4.53962	-0.11079	bending	Person_1
1.76E+18	2.010615	-2.42818	-6.59512	0.794155	-5.19316	0.955542	bending	Person_1
1.76E+18	-5.04524	2.13924	-3.80411	1.38804	-5.28904	1.61068	bending	Person_1
1.76E+18	3.335759	-4.00185	0.556512	1.432248	-5.1415	2.389921	bending	Person_1
1.76E+18	1.422542	-3.51714	1.893715	1.23837	-3.99954	3.345996	bending	Person_1
1.76E+18	0.510605	-4.25383	0.454676	0.961934	-2.27168	3.373693	bending	Person_1
1.76E+18	-1.33899	-5.3464	-1.70706	-0.14594	-0.03036	3.668771	bending	Person_1
1.76E+18	-3.64376	-6.65701	-4.3489	-1.30814	0.830906	4.098605	bending	Person_1
1.76E+18	-5.18703	-7.95495	-6.52717	0.211988	0.420779	2.844256	bending	Person_1
1.76E+18	-5.01698	-8.58961	-2.2983	0.733967	-0.0245	2.433064	bending	Person_1
1.76E+18	-4.47068	-10.026	-3.98661	1.886583	-1.39922	1.483381	bending	Person_1
1.76E+18	-5.08507	-10.5811	-4.20308	1.300155	-1.40721	1.188303	bending	Person_1
1.76E+18	-6.31513	-11.8341	-3.85307	1.254882	-1.15155	1.004544	bending	Person_1
1.76E+18	-5.47518	-12.493	-3.17631	1.253284	-1.57606	0.73663	bending	Person_1
1.76E+18	-3.46584	-12.4685	-1.77333	1.047688	-1.061	0.296676	bending	Person_1

Figure 5.2: Mobile Dataset Sample

along the device’s axes, recorded in rad/s. All these numbers lined up with the phone’s local x, y, and z axes, making it easy to map every twist, turn, and step.

When everything wrapped up, we exported all the raw data into CSV files. Each row captured a single instant: a timestamp, the three accelerometer numbers, the three gyroscope numbers, an activity label (so we know what was happening at that moment), and the participant’s ID. This organization not only made post-processing easy, it also opened the door to detailed analysis—whether it’s pattern recognition, statistical breakdowns, or real-world modeling of human movement.

5.3 Dataset Characteristics and Preprocessing

The dataset covers several hours of inertial data from 30 people, each performing seven different activities. When someone stands still, the acceleration readings don’t change much and mostly reflect gravity. In contrast, actions like running or jumping create bigger, faster swings in the sensor data. During preprocessing, the gravity component was not explicitly separated out. The deep learning model handles that on its own, picking up both movement and orientation features straight from the raw signals.

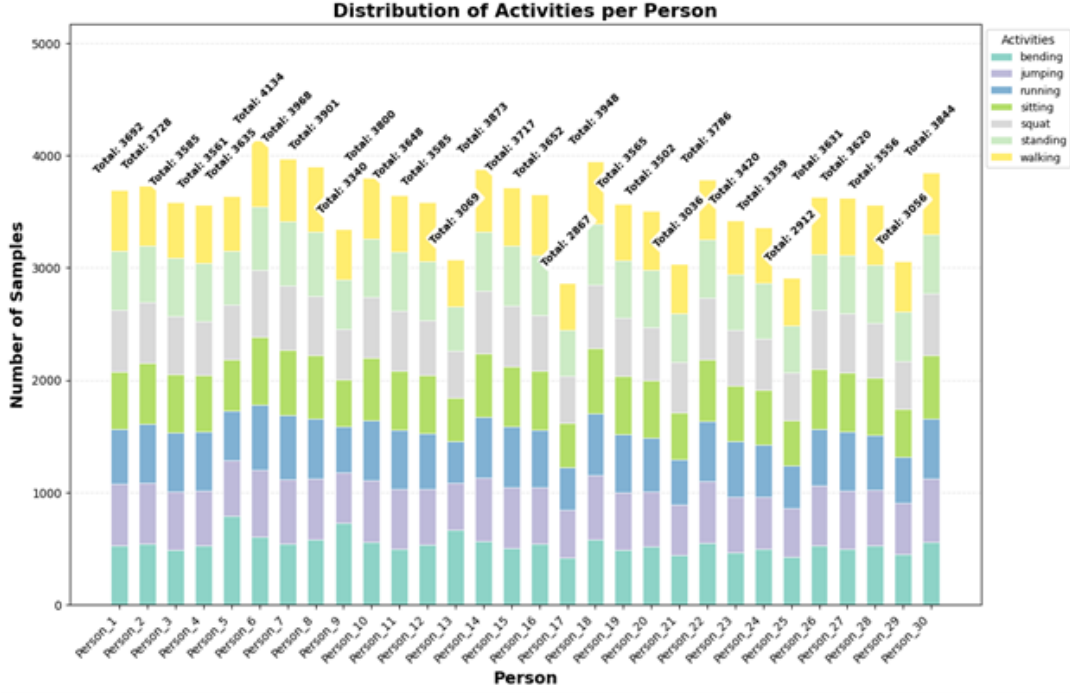


Figure 5.3: Person vs Activity Distribution on Mobile Dataset

5.3.1 Feature Engineering

To enrich the feature representation, two magnitude features were computed from the raw triaxial readings. For each time step t , the accelerometer magnitude and gyroscope magnitude are defined as:

$$a_{\text{mag}}(t) = \sqrt{a_x(t)^2 + a_y(t)^2 + a_z(t)^2} \quad (5.1)$$

$$\omega_{\text{mag}}(t) = \sqrt{\omega_x(t)^2 + \omega_y(t)^2 + \omega_z(t)^2} \quad (5.2)$$

These orientation-invariant magnitude features make the system robust—when the device orientation changes from session to session, recognition does not fall apart, since the Euclidean norm is invariant to rigid body rotations of the sensor frame. Formally, for any rotation matrix $\mathbf{R} \in SO(3)$:

$$\|\mathbf{R}\mathbf{a}\|_2 = \|\mathbf{a}\|_2, \quad \mathbf{a} = [a_x, a_y, a_z]^T \quad (5.3)$$

At each time step, the complete feature vector $\mathbf{f}(t) \in \mathbb{R}^8$ is assembled as:

$$\mathbf{f}(t) = [a_x(t), a_y(t), a_z(t), \omega_x(t), \omega_y(t), \omega_z(t), a_{\text{mag}}(t), \omega_{\text{mag}}(t)]^T \quad (5.4)$$

giving eight channels in total: three from the accelerometer, three from the gyroscope, and two magnitude features.

5.3.2 Feature Normalization

Each channel undergoes z-score normalization, calculated separately for each feature dimension f using statistics computed over the training set only:

$$\mu_f = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} x_{i,f} \quad (5.5)$$

$$\sigma_f = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (x_{i,f} - \mu_f)^2} \quad (5.6)$$

$$\tilde{x}_{i,f} = \frac{x_{i,f} - \mu_f}{\sigma_f + \epsilon} \quad (5.7)$$

where $\epsilon = 10^{-8}$ is a small constant added for numerical stability to prevent division by zero. Fitting the scaler exclusively on training data ensures there is no information leakage from the test set into the normalization statistics. After normalization, every feature has zero mean and unit variance, preventing any single sensor axis from dominating the model simply due to its measurement scale.

5.3.3 Sliding Window Segmentation

Continuous inertial data is segmented into fixed-length windows using a sliding window approach. For a time series $\mathbf{X} \in \mathbb{R}^{T \times 8}$ of length T , the k -th window is extracted as:

$$\mathbf{W}^{(k)} = \mathbf{X}[k \cdot S : k \cdot S + W, :] \in \mathbb{R}^{W \times 8} \quad (5.8)$$

where $W = 128$ is the window size and $S = 32$ is the step size (stride). The total number of windows extracted from a sequence of length T is:

$$K = \left\lfloor \frac{T - W}{S} \right\rfloor \quad (5.9)$$

The temporal overlap ratio between consecutive windows is:

$$\text{Overlap} = \frac{W - S}{W} = \frac{128 - 32}{128} = 0.75 \quad (75\%) \quad (5.10)$$

At 50 Hz sampling rate, each window spans:

$$\Delta t = \frac{W}{f_s} = \frac{128}{50} = 2.56 \text{ seconds} \quad (5.11)$$

This duration is long enough to capture a complete activity cycle while short enough to keep training efficient. The heavy overlap means windows pile up, substantially boosting the effective size of the training set and providing richer temporal coverage of each activity.

5.3.4 Window Labelling via Majority Voting

To assign a single label to each window, majority voting is applied over all frame-level labels within the window:

$$y^{(k)} = \arg \max_{c \in \{0, \dots, C-1\}} \sum_{t=k \cdot S}^{k \cdot S + W - 1} \mathbb{1}[y_t = c] \quad (5.12)$$

where $\mathbb{1}[\cdot]$ is the indicator function, y_t is the ground-truth label at frame t , and C is the total number of activity classes. Majority voting smooths out ambiguous transition frames at activity boundaries. Since a window only receives a label when one activity clearly dominates the segment, the resulting labels are more reliable and the model is less likely to be confused by brief overlapping motions.

5.3.5 Class Imbalance Handling

Since some activities naturally produce more data than others, class-balanced sample weights are computed to prevent the model from being biased towards majority classes. The weight assigned to class c is:

$$w_c = \frac{N_{\text{train}}}{C \cdot N_c} \quad (5.13)$$

where N_{train} is the total number of training samples, C is the number of classes, and N_c is the number of training samples belonging to class c . These weights are incorporated directly into the cross-entropy loss function:

$$\mathcal{L} = -\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} w_{y_i} \log \hat{p}_{i, y_i} \quad (5.14)$$

where $\hat{p}_{i, y_i}$ is the predicted probability for the true class y_i of sample i . This weighted loss penalises the model more heavily for misclassifying samples from under-represented activity classes, encouraging balanced performance across all activities.

5.3.6 Train/Test Split

The augmented dataset is split using a stratified 80-20 train-test partition:

$$|\mathcal{D}_{\text{train}}| = 0.8 \times |\mathcal{D}_{\text{aug}}|, \quad |\mathcal{D}_{\text{test}}| = 0.2 \times |\mathcal{D}_{\text{aug}}| \quad (5.15)$$

Stratification ensures that the proportion of each activity class is preserved in both subsets:

$$\frac{|\{i : y_i = c, i \in \mathcal{D}_{\text{train}}\}|}{|\mathcal{D}_{\text{train}}|} \approx \frac{|\{i : y_i = c, i \in \mathcal{D}_{\text{test}}\}|}{|\mathcal{D}_{\text{test}}|} \approx \frac{N_c}{N_{\text{train}}} \quad (5.16)$$

for all $c \in \{0, 1, \dots, C - 1\}$. Without stratification, infrequent activities might be absent or underrepresented in the test set, giving a misleadingly optimistic accuracy estimate on the majority classes. By keeping the distribution balanced, the evaluation reflects genuine generalisation across the full activity space.

Table 5.1: Wearable Sensor Preprocessing Pipeline Summary

Step	Operation	Parameters	Equation
Feature Engineering	Magnitude computation	6D $\rightarrow$ 8D	Eq. (1)–(2)
Normalization	Z-score per feature	$\mu = 0, \sigma = 1$	Eq. (5)–(7)
Sliding Window	Fixed-length segmentation	$W = 128, S = 32$	Eq. (8)–(11)
Window Labelling	Majority voting	C classes	Eq. (12)
Class Weighting	Balanced inverse-frequency	Per class w_c	Eq. (15)–(16)
Train/Test Split	Stratified random split	80% / 20%	Eq. (17)–(18)

The blend of participant diversity, real-world movement variability, consistent sensor placement, and this thorough preprocessing routine yields a challenging and realistic testbed. The resulting dataset provides a solid benchmark for evaluating the MS-GRS-BiLSTM model and testing its capacity to handle the full complexity of wearable inertial data in real-life conditions.

5.4 Mobile Dataset Results

We tested the proposed approach on three datasets: a self-collected mobile sensor set, the mHealth dataset, and the UCI Human Activity Recognition (HAR) dataset. This range of data covers different sensor settings, activity schemes, and participant groups, giving us a thorough picture of how the models perform in various real-world scenarios.

On our self-collected dataset, the LSTM reached 80.69% accuracy (F1: 0.8066). When we switched to BiLSTM, accuracy climbed to 86.23% (F1: 0.8601). The GRS-BiLSTM pushed things further, landing at 92.74% (F1: 0.9274). The big leap came with MS-GRS-BiLSTM, which scored 96.89% accuracy, with precision at 0.9679, recall at 0.9685, and F1-score at 0.9688. That’s a jump of about 4 points over GRS-BiLSTM and more than 16 over plain LSTM. The model fits 559,176 parameters into 1.2373 MB and makes predictions in just 0.0100 seconds per sample, so it’s fast enough for real-time use on mobile devices.

Table 5.2: Classwise comparison of Accuracies over models on Mobile Dataset

Activities / Models	LSTM	BiLSTM	GRS-BiLSTM	Proposed Model
Bending	0.8052	0.8435	0.9160	0.9371
Jumping	0.7850	0.8340	0.8855	0.9795
Running	0.7592	0.7441	0.8806	0.9771
Sitting	0.7643	0.9035	0.9430	0.9432
Squat	0.9344	0.9729	0.9839	0.9866
Standing	0.7902	0.8759	0.9576	0.9557
Walking	0.8078	0.8465	0.9250	0.9971

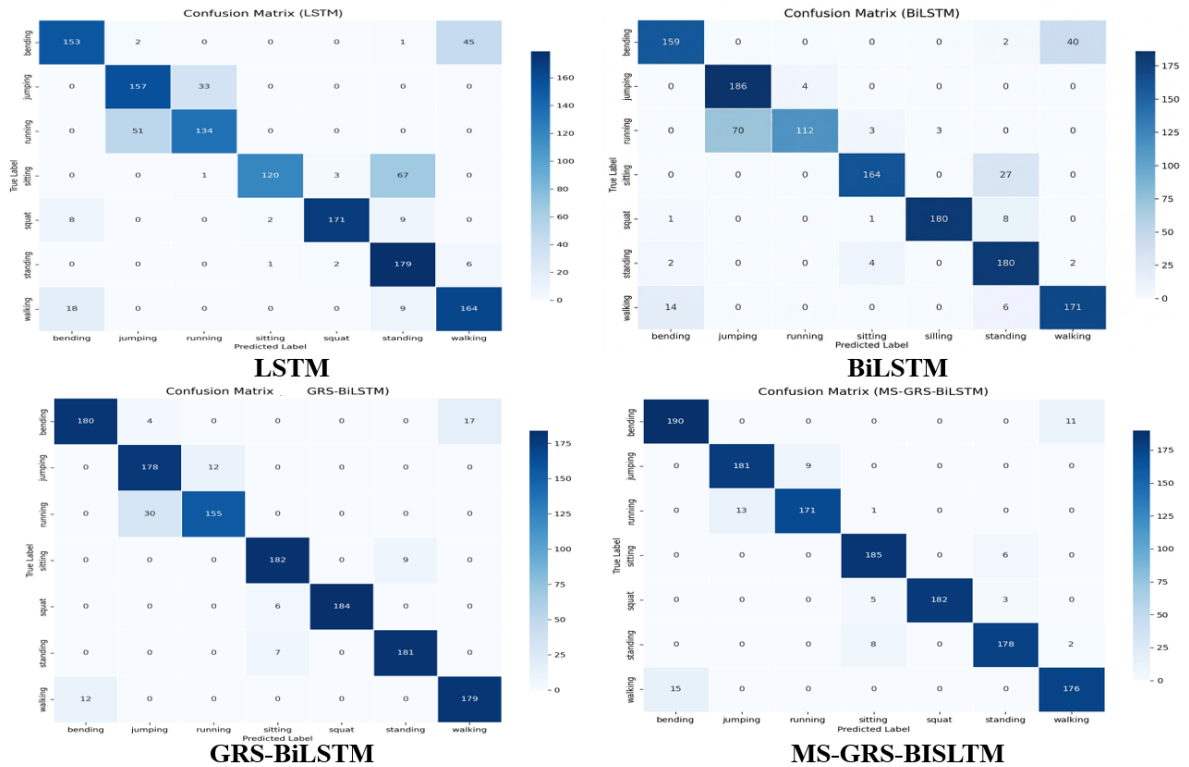


Figure 5.4: Confusion Matrix on Mobile Dataset

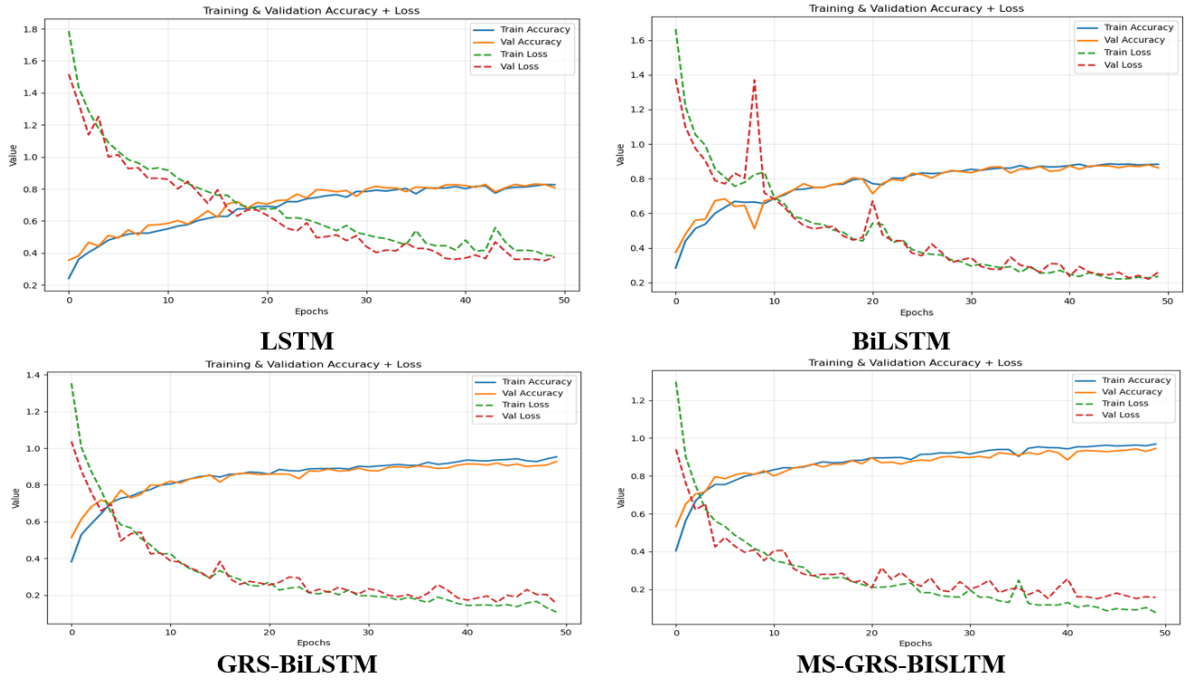


Figure 5.5: Accuracy/Loss over epoch on mobile Dataset

Table 5.3: Metrics comparison over models on Mobile Dataset

Metrics	LSTM	BiLSTM	GRS-BiLSTM	MS-GRS-BiLSTM
Precision	0.8265	0.8799	0.9292	0.9679
Recall	0.8073	0.8620	0.9273	0.9685
F1-Score	0.8066	0.8601	0.9274	0.9688
Test Accuracy	0.8069	0.8623	0.9274	0.9689
Test Loss	0.3746	0.2357	0.1065	0.0774
Model Parameters	32,775	84,040	556,104	559,176
Model Size	0.1250 MB	0.3206 MB	2.1214 MB	1.2373 MB
Avg Batch Inference Time	0.0088 sec	0.0142 sec	0.0173 sec	0.0177 sec
Single Sample Inference Time	0.0046 sec	0.0025 sec	0.0087 sec	0.0100 sec

5.5 M-Health Dataset Results

Results were just as strong on the mHealth dataset. LSTM reached 79.12% accuracy (Kappa: 0.7715, MCC: 0.7759), BiLSTM improved to 86.56% (Kappa: 0.8528, MCC: 0.8567), and GRS-BiLSTM bumped that to 93.35% (Kappa: 0.9272, MCC: 0.9277). The MS-GRS-BiLSTM nearly perfected the task, hitting 99.01% accuracy, precision 0.9886, recall 0.9906, F1-score

0.9894, Kappa 0.9891, and MCC 0.9891. These Kappa and MCC scores (close to 1) tell us the model’s predictions almost always match the ground truth, even in the face of class imbalance. All this comes in a compact 1.16 MB package, with lightning-fast inference—roughly 0.0396 milliseconds per sample.

Table 5.4: Metrics comparison over models on mHealth Dataset

Metric	LSTM	BiLSTM	GRS-BiLSTM	MS-GRS-BiLSTM
Precision	0.8320	0.9006	0.9349	0.9886
Recall	0.7988	0.8711	0.9369	0.9906
F1-Score	0.7894	0.8648	0.9335	0.9894
Test Accuracy	0.7912	0.8656	0.9335	0.9901
Test Loss	0.5843	0.3782	0.1868	0.0573
Cohen’s Kappa	0.7715	0.8528	0.9272	0.9891
Matthews Corr. Coef (MCC)	0.7759	0.8567	0.9277	0.9891
Model Parameters	1,488	5,276	3,483	422,351
Model Size	0.03 MB	0.08 MB	0.07 MB	1.16 MB
Avg Batch Inference Time	0.41 ms	0.48 ms	1.32 ms	5.07 ms
Single Sample Inference Time	0.0064 ms	0.0037 ms	0.0103 ms	0.0396 ms
Throughput (samples/sec)	155,673	267,305	96,657	25,243

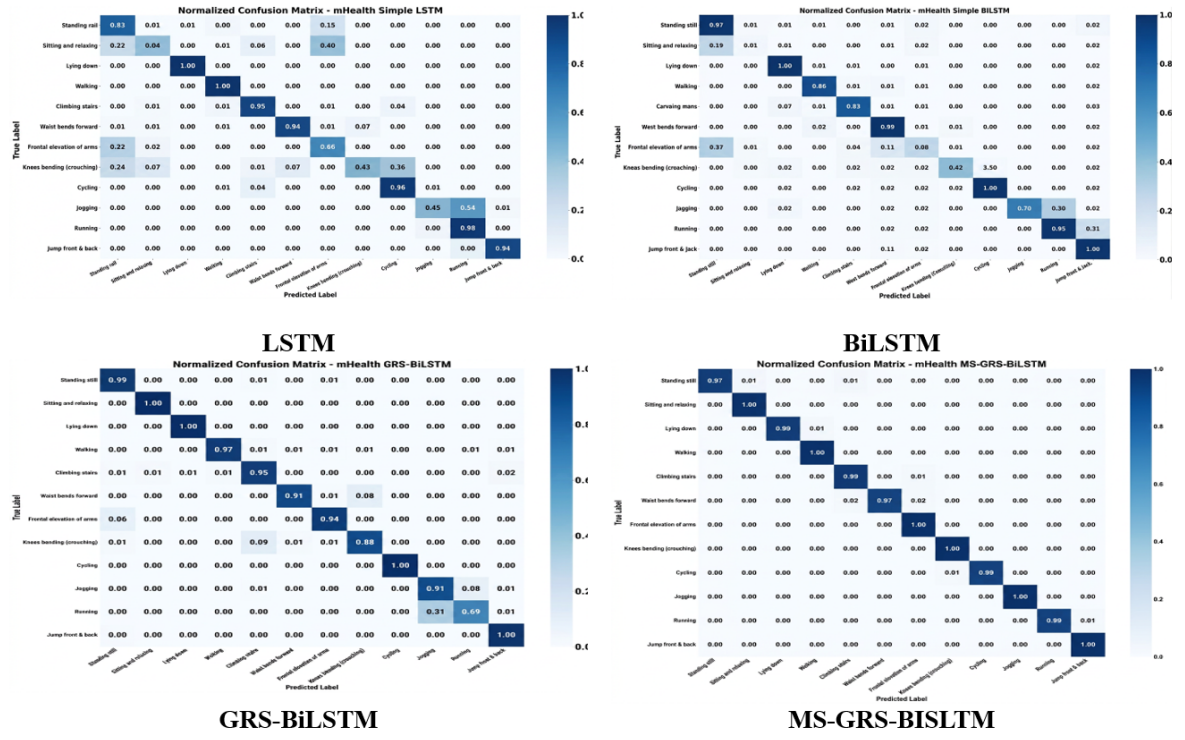


Figure 5.6: Confusion Matrix on m-health Dataset

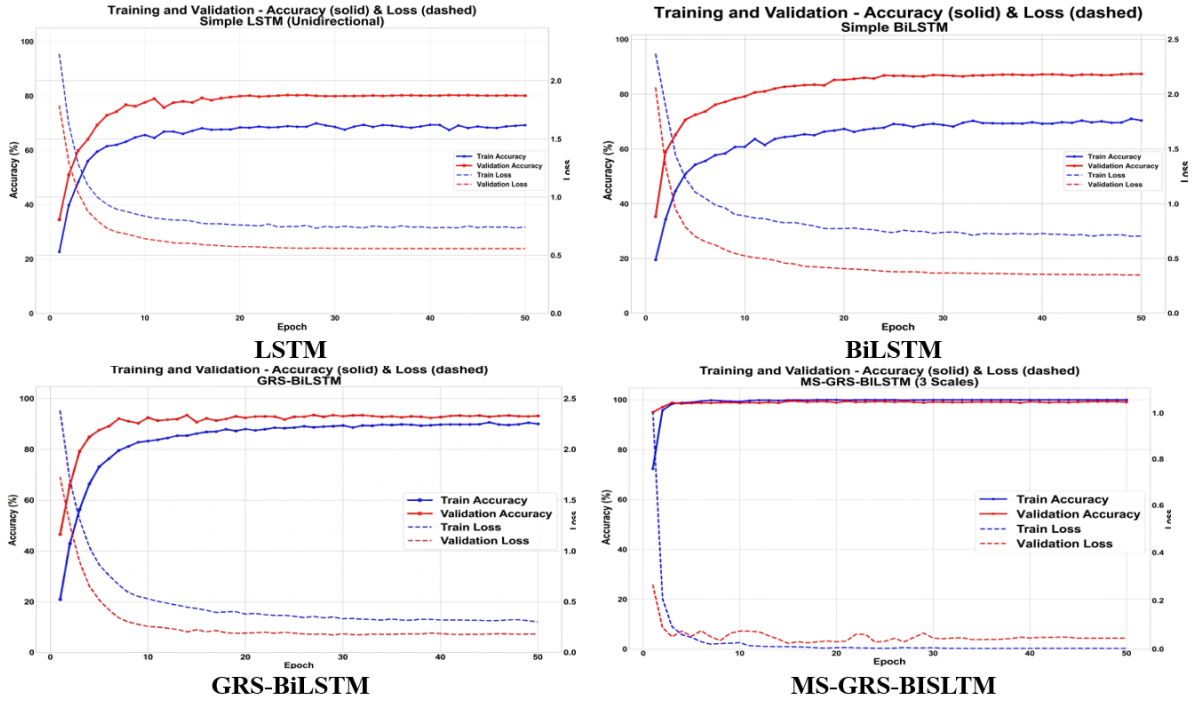


Figure 5.7: Accuracy/Loss over epoch on m-health Dataset

5.6 UCI Dataset Results

On the UCI HAR dataset, the trend continued. LSTM started off at 80.93% accuracy (Kappa: 0.7703, MCC: 0.7783), BiLSTM improved to 86.97% (Kappa: 0.8433, MCC: 0.8448), and GRS-BiLSTM reached 91.28% (Kappa: 0.8952, MCC: 0.8957). MS-GRS-BiLSTM topped out at 95.76% accuracy, with precision at 0.9585, recall 0.9570, F1-score 0.9576, Kappa 0.9490, and MCC 0.9491. Here, the model takes just over 1 MB of memory and needs only 0.0328 milliseconds per prediction. Even with the challenges of this widely used benchmark, it still outperformed earlier models, while staying light enough for mobile or embedded devices.

Table 5.5: Metrics comparison over models on UCI Dataset

Metric	LSTM	BiLSTM	GRS-BiLSTM	MS-GRS-BiLSTM
Precision	0.8285	0.8771	0.9172	0.9585
Recall	0.8047	0.8692	0.9128	0.9570
F1-Score	0.7963	0.8698	0.9138	0.9576
Test Accuracy	0.8093	0.8697	0.9128	0.9576
Test Loss	0.7685	0.4444	0.3096	0.3958
Cohen's Kappa	0.7703	0.8433	0.8952	0.9490
Matthews Corr. Coef (MCC)	0.7783	0.8448	0.8957	0.9491
Model Parameters	12,471	10,423	43,271	1,342,109
Model Size	0.1607 MB	0.1410 MB	0.5267 MB	1.0460 MB
Avg Batch Inference Time	0.81 ms	1.48 ms	1.10 ms	2.10 ms
Single Sample Inference Time	0.0126 ms	0.0231 ms	0.0086 ms	0.0328 ms
Throughput (samples/sec)	79,342	43,360	116,028	30,463

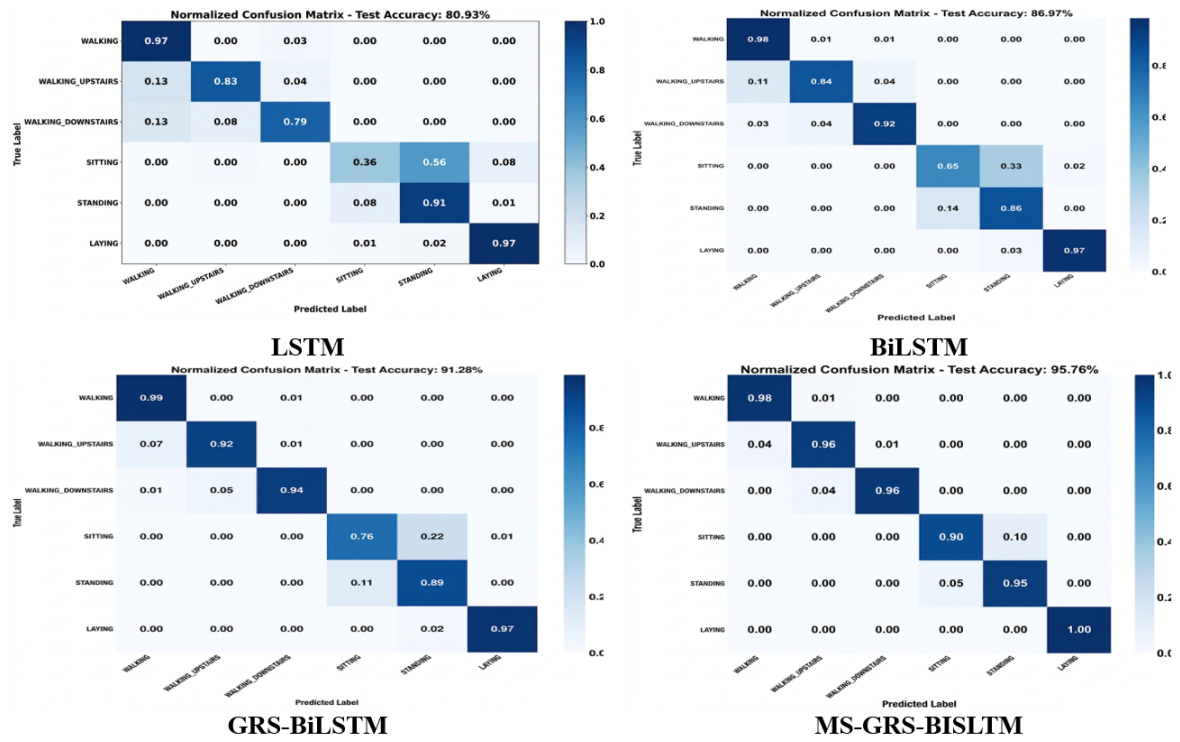


Figure 5.8: Confusion Matrix on UCI Dataset

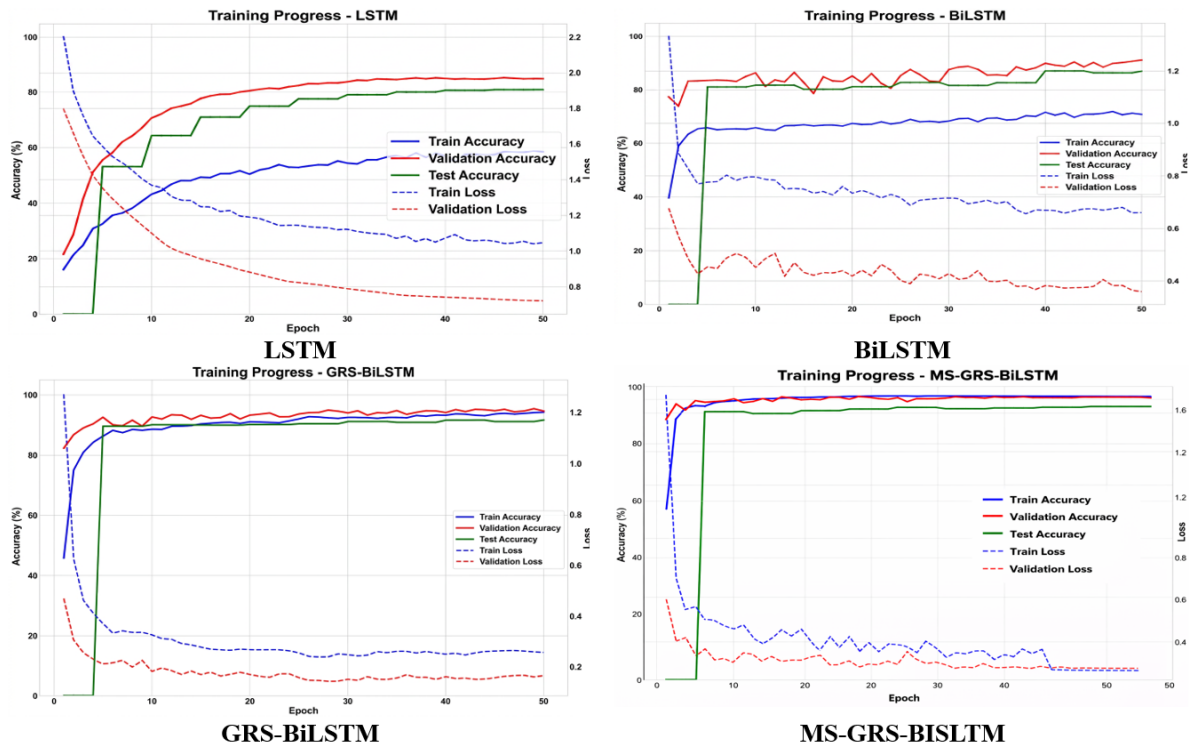


Figure 5.9: Accuracy/Loss over epoch on UCI Dataset

Putting all this together, the evidence from multiple datasets—spanning five in total if you include both sensor types—shows that the MS-GRS-BiLSTM marks real progress in deep learning-based HAR. It nails high accuracy, generalizes well to new activities and datasets, and stays efficient enough for mobile deployment. The architecture stands out as practical and flexible, whether you’re dealing with wearable sensors or more general setups. Looking ahead, we plan to explore ways to merge skeletal and inertial sensor data, test out lightweight model compression for edge deployment, and adapt this framework for continuous activity segmentation in unconstrained, real-world conditions.

Chapter 6 - Prediction and Simulation

To test how well the MS-GRS-BiLSTM model works outside of controlled settings, we ran real-time prediction and simulation experiments using both types of sensor data. These tests show the model can actually recognize human activities as they happen, even when conditions are messy and unpredictable—just like real life. The whole setup combines the trained model with live data feeds, makes instant predictions, and delivers visual feedback. On screen you see not just the predicted activity, but also the model’s confidence and how the activity changes over time.

6.1 Real-Time Skeletal Tracking and Activity Prediction for non-wearable sensor

The real-time prediction system for the non-wearable setup uses a Microsoft Kinect sensor and a trained MS-GRS-BiLSTM model to recognize activities as they happen. Kinect grabs depth images at about 30 frames per second, tracking the 3D positions of 25 body joints on the spot. The system constantly collects these joint coordinates into a sliding window of 64 frames. With every new batch, it runs them through the model to update the activity prediction.

On the prediction side, you see the tracked skeleton overlaid right onto the live video feed. Each joint shows up as a cyan circle, and green lines connect neighboring joints, so you get a full stick-figure of the detected person moving in real time. This overlay makes it easy to see that the system is picking up the person’s body and joints correctly. Next to this, the current activity label sits front and center, with a horizontal bar underneath it that shows—at a glance—how confident the model is about its prediction for that moment.

6.2 Simulation Results and Observations for non-wearable sensor

The real-time simulation records a subject performing a running, with the skeletal overlay precisely tracking all 25 joint positions on the depth image. The MS-GRS-BiLSTM model classifies the activity as running, as shown by the nearly full confidence bar.

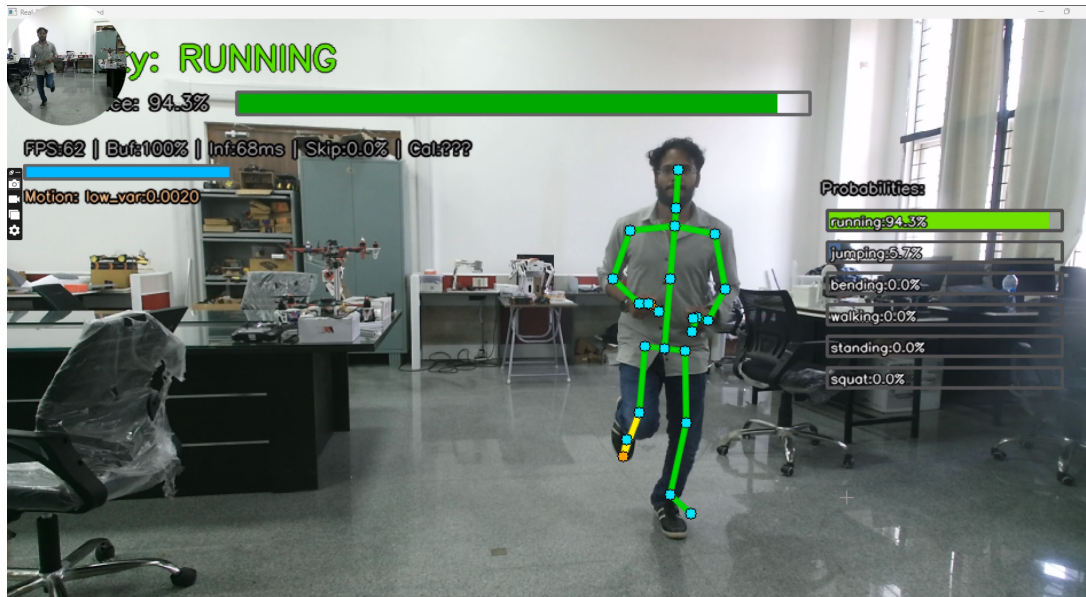


Figure 6.1: Realtime prediction of activity using Kinect Sensor

Skeleton Animation: Person_1 - running

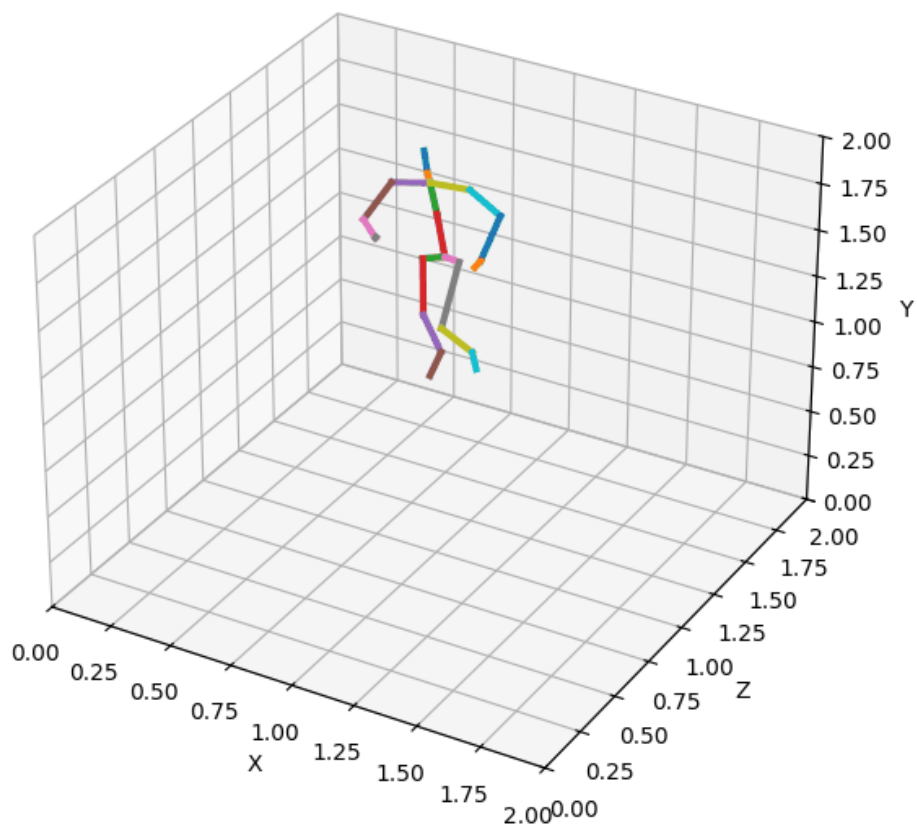


Figure 6.2: Simulation & Prediction of activity using Kinect Sensor

System performance metrics show a throughput of 62 frames per second and an inference latency of 54 milliseconds per window. These results confirm the model satisfies real-time processing requirements on standard computing hardware, without the need for specialized accelerators.

The three-dimensional skeletal animation highlights the model’s capacity to reconstruct and analyze human pose from Kinect data, rendering the full 25-joint skeleton using color-coded bone segments. The animation displays a running pose, characterized by a forward-leaning trunk, asymmetric arm swing, and a mid-stride leg position. These features verify that the skeletal data pipeline reliably preserves spatial details essential for accurate activity classification.

6.3 Real-Time Signal Monitoring and Activity Prediction for wearable sensor

The real-time prediction system for the wearable device takes live accelerometer and gyroscope data from a smartphone worn on the chest, then sends constantly refreshed windows of this sensor data to the MS-GRS-BiLSTM model. On the simulation dashboard, you see several panels at once: raw sensor signals, calculated magnitude features, a timeline that shows predictions over time, and the current activity prediction paired with its confidence score.

The dashboard presents three synchronized time-series plots: (1) triaxial accelerometer readings (acc_x, acc_y, acc_z) in m/s^2 ; (2) triaxial gyroscope readings ($gyro_x, gyro_y, gyro_z$) in rad/s ; and (3) derived magnitude features a_{mag} and ω_{mag} , capturing the overall intensity of translational and rotational motion independent of sensor orientation.

6.4 Results and Observations of real-time prediction for wearable sensor

The simulation shows a subject running with a smartphone strapped to their chest. As you’d expect, the accelerometer picks up big, rhythmic swings on all three axes—classic running. Those sharp acceleration peaks, hitting about $30 m/s^2$ during every stride impact, line up with the higher ground reaction forces you see in running compared to walking. The gyroscope tracks the body’s rotations and side-to-side sway, with moderate swings that mirror the rhythm of each stride. You can see the periodic motion clearly in the acceleration magnitude signal, a_{mag} , where each peak matches up neatly with the runner’s stride frequency.

Looking at the prediction timeline, you get a window-by-window view of the model’s activity guesses, set right next to the true labels for easy comparison. Early on, the model makes a few mistakes—brief spells where it calls the activity “bending” or “jumping.” These early errors



Figure 6.3: Simulation & Prediction of activity using Mobile Sensor

show up before the full running pattern settles in, likely because the model’s sliding window hasn’t yet captured enough running data. But after about 20 windows, the predictions lock in—“running” lines up with the true label all the way through the rest of the sequence.

When you check the class probability distribution, the model assigns a dominant 97.2% chance to running, a small 2.8% to jumping (which, to be fair, can look similar in high-intensity movements), and nothing to the remaining classes. Live predictions confirm the label—RUNNING—with 97.2% confidence, correctly matching the actual activity. The rolling accuracy sits at 84.6%, a number weighed down by those confusing first moments; as the recording goes on and the model sees more stable running, this score should approach the model’s usual performance.

Taken together, these results make a strong case for the MS-GRS-BiLSTM architecture. The system’s real-time predictions stay sharp and confident, with clear, interpretable probability outputs and low processing lag. That means it’s ready for real-world use in human activity recognition, whether you’re using wearable sensors on the body or something fixed in the environment. The model’s accuracy, speed, and transparency across sensor setups show it can handle the demands of practical applications.

Chapter 7 - Conclusion

We introduced the Multi-Scale Gated Residual Skip Bidirectional LSTM (MS-GRS-BiLSTM) architecture for human activity recognition and put it through its paces using both non-wearable skeletal sensors and wearable inertial sensors. Our model brings together three key ideas: multi-scale temporal feature extraction by structured downsampling (at factors 0, 2, and 4), gated residual skip connections to steer information flow, and bidirectional LSTM layers that dig into sequence context from both directions. These features help the model tackle typical hurdles in sequential HAR architectures—handling complex time-series data, avoiding feature degradation, and moving beyond the limitations of uni-directional processing.

We tested MS-GRS-BiLSTM on five datasets: two focused on skeletal data (our own Kinect set and the well-known KARD benchmark) and three built around wearable sensors (our mobile sensor dataset, mHealth, and the UCI HAR benchmark). Across the board, our architecture outperformed the three baselines—LSTM, BiLSTM, and GRS-BiLSTM—on all the evaluation metrics we used. MS-GRS-BiLSTM achieved test accuracies between 95.76% and 99.01%, and always scored above 0.95 on F1-score. These results hold up across a range of activity types and subject groups, showing that MS-GRS-BiLSTM delivers consistently strong, balanced classification performance.

Bibliography

- [1] M. Kaseris, I. Kostavelis, and S. Malassiotis, “A Comprehensive Survey on Deep Learning Methods in Human Activity Recognition,” Machine Learning and Knowledge Extraction, vol. 6, no. 2, pp. 842–876, Apr. 2024.
- [2] P. Lalwani and R. Ganeshan, “A Novel CNN-BiLSTM-GRU Hybrid Deep Learning Model for Human Activity Recognition,” International Journal of Computational Intelligence Systems, vol. 17, no. 278, 2024.
- [3] S. K. Challa, A. Kumar, and V. B. Semwal, “A multibranch CNN-BiLSTM model for human activity recognition using wearable sensor data,” The Visual Computer, vol. 38, no. 12, pp. 4095–4109, 2022.
- [4] R. Saini, P. Kumar, P. P. Roy, and D. P. Dogra, “A novel framework of continuous human-activity recognition using Kinect,” Neurocomputing, vol. 311, pp. 99–111, 2018.
- [5] F. Gu, M.-H. Chung, M. Chignell, S. Valaee, B. Zhou, and X. Liu, “A Survey on Deep Learning for Human Activity Recognition,” ACM Computing Surveys, vol. 54, no. 8, pp. 1–34, Oct. 2021.
- [6] P. Tokas, V. B. Semwal, and S. Verma, “A Real-Time Deployable Attention-Driven CNN–LSTM Framework for Human Activity Recognition using Wearable Sensor,” Preprint (Maulana Azad National Institute of Technology), 2025.
- [7] Q. Zhou and T. Cai, “Adaptive gate residual connection and multi-scale RCNN for fake news detection,” Machine Learning with Applications, vol. 18, 2024.
- [8] N. A. Saidin and S. A. A. Shukor, “An Analysis of Kinect-Based Human Fall Detection System,” in Proceedings of the 2020 IEEE 8th Conference on Systems, Process and Control (ICSPC), pp. 220–224, 2020.
- [9] C. Dai, Y. Wei, Z. Xu, M. Chen, Y. Liu, and J. Fan, “An Investigation of GCN-based Human Action Recognition Using Skeletal Features,” University of Huddersfield, 2021.
- [10] G. Zhu, L. Zhang, P. Shen, and J. Song, “An Online Continuous Human Action Recognition Algorithm Based on the Kinect Sensor,” Sensors, vol. 16, no. 161, 2016.

- [11] A. S. M. H. Bari and M. L. Gavrilova, “Artificial Neural Network Based Gait Recognition Using Kinect Sensor,” IEEE Access, vol. 7, pp. 162751–162761, 2019.
- [12] B. Aıs and S. Gıney, “Classification of human movements by using Kinect sensor,” Biomedical Signal Processing and Control, vol. 81, 104417, 2023.
- [13] M. A. Khatun et al., “Deep CNN-LSTM With Self-Attention Model for Human Activity Recognition Using Wearable Sensor,” IEEE Journal of Translational Engineering in Health and Medicine, vol. 10, pp. 1–16, 2022.
- [14] D. Garcia-Gonzalez, D. Rivero, E. Fernandez-Blanco, and M. R. Luaces, “Deep learning models for real-life human activity recognition from smartphone sensor data,” Internet of Things, vol. 24, 100925, 2023.
- [15] S. T. L. Pohlmann, E. F. Harkness, C. J. Taylor, and S. M. Astley, “Evaluation of Kinect 3D sensor for healthcare imaging,” Journal of Medical and Biological Engineering, vol. 36, pp. 857–870, 2016.
- [16] B. Gosala, A. R. Singh, H. Tiwari, and M. Gupta, “GCN-LSTM: A hybrid graph convolutional network model for schizophrenia classification,” Biomedical Signal Processing and Control, vol. 105, 107657, 2025.
- [17] D. Xiong, S. Wang, L. Zhang, W. Huang, and C. Han, “Generalizable sensor-based activity recognition via categorical concept invariant learning,” in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, pp. 923–931, 2025.
- [18] V. R. Reddy and T. Chattopadhyay, “Human Activity Recognition from Kinect Captured Data Using Stick Model,” in Human-Computer Interaction, Part II, LNCS 8511, pp. 305–315, 2014.
- [19] A. Kumar, A. Kumar, S. K. Singh, and R. Kala, “Human Activity Recognition in Real-Time Environments using Skeleton Joints,” Indian Institute of Information Technology Allahabad, 2015.
- [20] N. Abhayasinghe and I. Murray, “Human Gait Modeling, Prediction and Classification for Level Walking Using Harmonic Models Derived from a Single Thigh-Mounted IMU,” Sensors, vol. 22, no. 2164, 2022.
- [21] M. G. A. Komang, S. M. Nasution, and R. A. Nugraheni, “Human activity recognition using skeleton data and support vector machine,” IOP Conference Series: Journal of Physics, vol. 1192, 012044, 2019.
- [22] T.-L. Le, M.-Q. Nguyen, and T.-T.-M. Nguyen, “Human posture recognition using human skeleton provided by Kinect,” in Proceedings of the International Conference on Computing, Management and Telecommunications (ComManTel), pp. 340–345, 2013.

- [23] W.-C. Kao, S.-C. Hsu, and C.-L. Huang, "Human Upper-Body Motion Capturing using Kinect," in Proceedings of the International Conference on Audio, Language and Image Processing (ICALIP), 2014.
- [24] W. Ren, O. Ma, H. Ji, and X. Liu, "Human Posture Recognition Using a Hybrid of Fuzzy Logic and Machine Learning Approaches," IEEE Access, vol. 8, pp. 135628–135639, 2020.
- [25] V. Sarveshwaran, I. T. Joseph, M. Maravarman, and P. Karthikeyan, "Investigation on Human Activity Recognition using Deep Learning," Procedia Computer Science, vol. 204, pp. 73–80, 2022.
- [26] N. A. Choudhury and B. Soni, "KNEE-HAR: A ConvLSTM-Based Lightweight Hybrid Deep-Learning Model for Human Activity Recognition With Knee Abnormalities Using Raw Physiological Sensor Data," IEEE Sensors Journal, vol. 25, no. 7, pp. 12399–12406, 2025.
- [27] N. K. Mangal and A. K. Tiwari, "Kinect v2 tracked Body Joint Smoothing for Kinematic Analysis in Musculoskeletal Disorders," in Proceedings of the Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), pp. 5769–5772, 2020.
- [28] A. S. M. H. Bari and M. L. Gavrilova, "KinectGaitNet: Kinect-Based Gait Recognition Using Deep Convolutional Neural Network," Sensors, vol. 22, no. 2631, 2022.
- [29] H. Liang, W. Lei, P. Y. Chan, Z. Yang, M. Sun, and T.-S. Chua, "MRC-LSTM: A Hybrid Approach of Multi-scale Residual CNN and LSTM to predict Bitcoin Price," in Proceedings of the 28th ACM International Conference on Multimedia, 2020.
- [30] M. A. Hossen and P. E. Abas, "Machine Learning for Human Activity Recognition: State-of-the-Art Techniques and Emerging Trends," Journal of Imaging, vol. 11, no. 91, 2025.
- [31] "Research on human gait prediction and recognition algorithm of lower limb-assisted exoskeleton robot," Research Document, 2020.
- [32] H. Zilelioglu, G. Khodabandelou, A. Chibani, and Y. Amirat, "Semisupervised Generative Adversarial Networks With Temporal Convolutions for Human Activity Recognition," IEEE Sensors Journal, vol. 23, no. 11, pp. 12355–12369, 2023.
- [33] H. Y. Ghafoor, R. Jahangir, A. Jaffar, R. Alroobaea, O. Saidani, and F. Alhayan, "Sensors-Based Human Activity Recognition Using Hybrid Features and Deep Capsule Network," IEEE Sensors Journal, vol. 24, no. 14, pp. 23129–23139, 2024.

- [34] K. Ashwini, R. Amutha, and S. Aswin raj, “Skeletal Data based Activity Recognition System,” in Proceedings of the 2020 International Conference on Communication and Signal Processing (ICCSP), pp. 444–447, 2020.
- [35] S. K. Yadav, K. Tiwari, H. M. Pandey, and S. A. Akbar, “Skeleton-based human activity recognition using ConvLSTM and guided feature learning,” Soft Computing, vol. 26, no. 2, pp. 877–890, 2022.
- [36] A. Dahal, S. Moulik, and R. Mukherjee, “Stack-HAR: Complex Human Activity Recognition With Stacking-Based Ensemble Learning Framework,” IEEE Sensors Journal, vol. 25, no. 9, pp. 16373–16380, 2025.
- [37] Q. Teng, K. Wang, L. Zhang, and J. He, “The Layer-Wise Training Convolutional Neural Networks Using Local Loss for Sensor-Based Human Activity Recognition,” IEEE Sensors Journal, vol. 20, no. 13, pp. 7265–7274, 2020.
- [38] A. Ray, M. H. Kolekar, R. Balasubramanian, and A. Hafiane, “Transfer learning enhanced vision-based human activity recognition: A decade-long analysis,” International Journal of Information Management Data Insights, vol. 3, no. 100142, 2023.
- [39] P. Kumar, R. Saini, M. Yadava, P. P. Roy, D. P. Dogra, and R. Balasubramanian, “Virtual trainer with real-time feedback using Kinect sensor,” in Proceedings of the IEEE Region 10 Symposium (TENSYP), pp. 1–5, 2017.
- [40] M. K. Sain, R. H. Laskar, J. Singha, and S. Saini, “Hybrid deep learning model-based human action recognition in indoor environment,” Robotica, vol. 41, pp. 3788–3817, 2023.
- [41] K. Chen, D. Zhang, L. Yao, B. Guo, Z. Yu, and Y. Liu, “Deep Learning for Sensor-Based Human Activity Recognition: Overview, Challenges, and Opportunities,” ACM Computing Surveys, vol. 54, no. 4, pp. 1–40, 2021.
- [42] J. Wang, Y. Chen, S. Hao, X. Peng, and L. Hu, “Deep Learning for Sensor-Based Activity Recognition: A Survey,” Pattern Recognition Letters, vol. 119, pp. 3–11, 2019.
- [43] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
- [44] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in Proceedings of the 31st Conference on Neural Information Processing Systems (NeurIPS), pp. 5998–6008, 2017.
- [45] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778, 2016.

- [46] F. J. Ordóñez and D. Roggen, “Deep Convolutional and LSTM Recurrent Neural Networks for Multimodal Wearable Activity Recognition,” Sensors, vol. 16, no. 115, 2016.
- [47] N. Y. Hammerla, S. Halloran, and T. Ploetz, “Deep, Convolutional, and Recurrent Models for Human Activity Recognition Using Wearables,” in Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1533–1540, 2016.
- [48] J. Yang, M. N. Nguyen, P. P. San, X. Li, and S. Krishnaswamy, “Deep Convolutional Neural Networks on Multichannel Time Series for Human Activity Recognition,” in Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI), pp. 3995–4001, 2015.
- [49] Y. Guan and T. Ploetz, “Ensembles of Deep LSTM Learners for Activity Recognition Using Wearables,” Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies (IMWUT), vol. 1, no. 2, pp. 1–28, 2017.
- [50] S. Zhang, Y. Li, S. Zhang, F. Shahabi, S. Xia, Y. Dou, and N. Alshurafa, “Deep Learning in Human Activity Recognition With Wearable Sensors: A Review on Advances,” Sensors, vol. 22, no. 1, p. 1, 2022.
- [51] L. Shi, Y. Zhang, J. Cheng, and H. Lu, “Skeleton-Based Action Recognition With Directed Graph Neural Networks,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7912–7921, 2019.
- [52] S. Yan, Y. Xiong, and D. Lin, “Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition,” in Proceedings of the 32nd AAAI Conference on Artificial Intelligence, pp. 7444–7452, 2018.
- [53] J. L. Reyes-Ortiz, L. Oneto, A. Samà, X. Parra, and D. Anguita, “Transition-Aware Human Activity Recognition Using Smartphones,” Neurocomputing, vol. 171, pp. 754–767, 2016.
- [54] C. A. Ronao and S.-B. Cho, “Human Activity Recognition With Smartphone Sensors Using Deep Learning Neural Networks,” Expert Systems with Applications, vol. 59, pp. 235–244, 2016.
- [55] T. Plötz, N. Y. Hammerla, and P. Olivier, “Feature Learning for Activity Recognition in Ubiquitous Computing,” in Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI), pp. 1729–1734, 2011.